

Spring & Spring Boot — The Complete Guide

Before Spring: Java Evolution, Servlets, JDBC, Hibernate

Spring Core • Spring MVC • Spring Boot • Transactions • Testing • Caching/Redis

Production-Level Implementation, Real Organization Project Structures, Interview Q&A

A practical, simple-language, end-to-end reference for interview preparation and real-world implementation

Table of Contents

1. Before Spring — The Evolution of Java Enterprise Development
2. Introduction to Servlets — What They Are and Their Problems
3. JDBC — Direct Database Access Before ORM
4. Hibernate & ORM Evolution
5. Introduction to Spring Framework — Why It Was Created
6. Introduction to Spring MVC
7. Introduction to Spring Boot
8. Struts vs Spring vs Spring MVC vs Spring Boot — Full Comparison
9. Project Structure — Plain Servlet/JSP Application
10. Project Structure — Spring MVC Application
11. Project Structure — Spring Boot Application (Real Organization Layout)
12. Spring Core — The IOC Container
13. Dependency Injection — Types and Real Examples
14. Spring Bean Lifecycle
15. Spring Bean Scopes
16. Autowiring in Spring
17. Configuration — XML vs Java Config vs Annotations
18. Spring Annotations — The Complete Reference
19. Spring Expression Language (SpEL)
20. AOP — Aspect Oriented Programming
21. Spring MVC Architecture — The Full Picture
22. DispatcherServlet Explained in Depth
23. Controllers in Spring MVC
24. View Resolver
25. Request Mapping — Full Reference
26. Model and ModelMap
27. Form Handling in Spring MVC
28. Data Binding
29. Validation in Spring MVC
30. Exception Handling in Spring MVC
31. Interceptors
32. JSP Integration & Thymeleaf Integration
33. Spring Boot — Auto Configuration Explained
34. Starter Dependencies
35. Application Properties / YAML Configuration
36. Spring Profiles
37. Spring Boot Actuator
38. Embedded Tomcat
39. CommandLineRunner & ApplicationRunner
40. @ConfigurationProperties
41. Logging in Spring Boot
42. Spring Transactions — @Transactional Deep Dive
43. Transaction Propagation

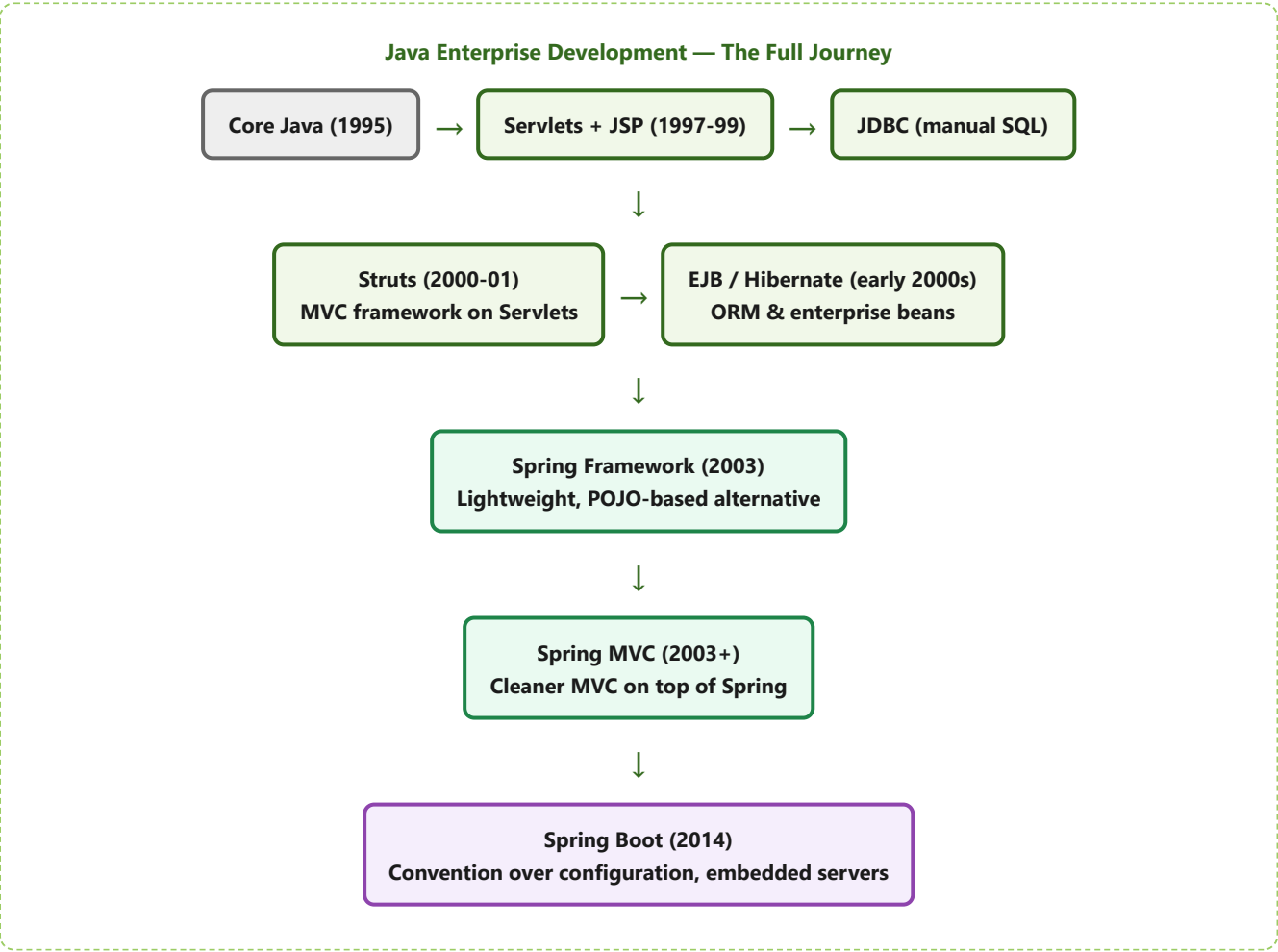
- 44. Transaction Isolation Levels
- 45. Rollback Handling
- 46. Distributed Transactions
- 47. Spring Testing — JUnit 5
- 48. Mockito
- 49. MockMvc
- 50. Integration Testing in Spring Boot
- 51. TestContainers
- 52. Caching in Spring Boot — Complete Deep Dive
- 53. Redis with Spring Boot — Complete Deep Dive
- 54. How We Develop Spring Boot Applications in Real-Time (Organization Level)
- 55. Common Spring/Spring Boot Issues, Problems & Solutions
- 56. Most Important Interview Topics — Deep Dive Recap
- 57. Interview Questions & Answers (Basic → Advanced → Scenario Based)
- 58. Final Summary & Cheat Sheet

1. Before Spring — The Evolution of Java Enterprise Development

1.1 The Starting Point — Plain Java (Core Java)

Java began as a general-purpose language — classes, objects, methods, no built-in concept of "web application." To build a web application in early Java, developers needed additional specifications and containers on top of Core Java.

1.2 The Full Evolution Timeline



1.3 Why Each Step Existed — Real Problems Solved

Technology	Problem It Solved	New Problem It Introduced
Servlets	Let Java handle HTTP requests dynamically (vs static HTML)	Mixing Java code and HTML output directly in one class was messy and hard to maintain
JSP	Let HTML-heavy pages embed Java logic more naturally	Business logic scattered directly inside view pages — poor separation of concerns

JDBC	Standard way to talk to relational databases from Java	Enormous boilerplate — manual connections, statements, result set mapping, resource cleanup
Struts	Brought proper MVC structure on top of raw Servlets/JSP	Heavy XML configuration, tightly coupled to the framework's own base classes
EJB (early versions)	Enterprise features — transactions, security, remote objects	Extremely heavyweight, required an EJB container, very slow development cycle, hard to unit test
Hibernate	Solved JDBC's boilerplate via Object-Relational Mapping	Still needed manual wiring/configuration; didn't solve the broader application architecture problem
Spring Framework	Lightweight, POJO-based, solved EJB's heaviness with Dependency Injection	Still required substantial XML/Java configuration to wire everything together
Spring Boot	Auto-configuration, embedded servers, starter dependencies — "just run it"	(Widely considered to have solved most prior pain points for typical applications)

Interview one-liner: "Each layer of the Java web stack's history exists because the previous layer solved one problem but created operational/boilerplate pain of its own — Spring Boot is the current end state of decades of the community trying to make Java web development simpler without losing enterprise-grade power."

2. Introduction to Servlets — What They Are and Their Problems

2.1 What Is a Servlet?

A Servlet is a Java class that runs inside a Servlet Container (like Apache Tomcat) and handles HTTP requests/responses directly — it's the foundational building block that every Java web framework (Struts, Spring MVC, Spring Boot) is ultimately built on top of.

2.2 A Real, Raw Servlet

```
public class AccountServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        String accountId = req.getParameter("id");
        Account account = accountDao.findById(Long.parseLong(accountId)); // manual lookup

        resp.setContentType("text/html");
        PrintWriter out = resp.getWriter();
        out.println("<html><body>");
        out.println("<h1>Balance: " + account.getBalance() + "</h1>"); // HTML hardcoded directly in Java!
        out.println("</body></html>");
    }
}

<!-- web.xml - manual URL mapping, no annotations in early Servlet versions -->
<servlet>
    <servlet-name>AccountServlet</servlet-name>
    <servlet-class>com.bank.AccountServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>AccountServlet</servlet-name>
    <url-pattern>/account</url-pattern>
</servlet-mapping>
```

2.3 Real Problems With Raw Servlets

- **HTML mixed with Java code** — printing HTML tags via `out.println()` is unmaintainable for any real UI.
- **One servlet class per URL**, or a lot of manual if/else routing logic inside a single servlet — no clean routing framework.
- **Verbose XML configuration** (`web.xml`) for every servlet mapping.
- **No separation of concerns** — a single class often handled request parsing, business logic, and HTML rendering.
- **No built-in dependency injection** — objects were manually instantiated (`new AccountDao()`) everywhere, making unit testing hard.

JSP (JavaServer Pages) was introduced specifically to fix the "HTML mixed with Java" problem by flipping the model — write HTML normally, embed small Java snippets only where needed (`<% %>` scriptlets) — but this created its own mess of business logic leaking into view pages, which MVC frameworks like Struts and later Spring MVC solved properly (Section 20).

3. JDBC — Direct Database Access Before ORM

3.1 What Is JDBC?

JDBC (Java Database Connectivity) is the standard low-level API for connecting Java applications to relational databases — every ORM (including Hibernate) is ultimately built on top of JDBC underneath.

3.2 Real Raw JDBC Code (What Developers Wrote Before Hibernate)

```
public Account findAccountById(Long id) {
    Connection conn = null;
    PreparedStatement stmt = null;
    ResultSet rs = null;
    try {
        conn = DriverManager.getConnection(DB_URL, USER, PASSWORD); // manual connection management
        stmt = conn.prepareStatement("SELECT * FROM account WHERE id = ?");
        stmt.setLong(1, id);
        rs = stmt.executeQuery();
        if (rs.next()) {
            Account account = new Account();
            account.setId(rs.getLong("id")); // manual column-by-column mapping
            account.setBalance(rs.getBigDecimal("balance"));
            account.setStatus(rs.getString("status"));
            return account;
        }
        return null;
    } catch (SQLException e) {
        throw new RuntimeException(e);
    } finally {
        // manual resource cleanup — easy to forget, causes connection leaks
        try { if (rs != null) rs.close(); } catch (SQLException ignored) {}
        try { if (stmt != null) stmt.close(); } catch (SQLException ignored) {}
        try { if (conn != null) conn.close(); } catch (SQLException ignored) {}
    }
}
```

3.3 Real Problems With Raw JDBC

Problem	Why It Hurts
Enormous boilerplate	The same connection/statement/resultset/cleanup pattern repeated for every single query
Manual object mapping	Every row-to-object mapping written by hand, error-prone and tedious at scale
Easy to leak resources	Forgetting to close a Connection/Statement/ResultSet in the finally block causes connection pool exhaustion over time
No object relationships	Fetching an Account with its Cards required manually writing and combining multiple queries by hand
SQL scattered everywhere	Raw SQL strings embedded directly in Java code across the codebase, hard to maintain/refactor

This exact pain — repetitive boilerplate and manual object mapping — is precisely what Hibernate/JPA (Section 4) and later Spring's `JdbcTemplate` /Spring Data JPA were built to eliminate.

4. Hibernate & ORM Evolution

4.1 What Is ORM (Object-Relational Mapping)?

ORM maps Java objects directly to database tables/rows, letting developers work with regular Java objects and let the framework generate and execute the actual SQL — Hibernate was the dominant Java ORM implementation that made this mainstream in the early-to-mid 2000s.

4.2 The Same Query, With Hibernate

```
@Entity
@Table(name = "account")
public class Account {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private BigDecimal balance;
    private String status;
    // getters/setters
}

// Fetching – Hibernate handles connection, SQL generation, and object mapping automatically
Session session = sessionFactory.openSession();
Account account = session.get(Account.class, id); // ONE line replaces the entire raw JDBC method from
Section 3.2
```

4.3 JPA vs Hibernate — A Common Point of Confusion

	JPA	Hibernate
What it is	A specification (interfaces/annotations) — <code>@Entity</code> , <code>@Id</code> , <code>EntityManager</code>	A concrete implementation of that specification (the actual working code)
Analogy	Like an interface	Like a class implementing that interface
Alternatives	—	EclipseLink, OpenJPA are other JPA implementations

Interview tip: When you use `@Entity` and `EntityManager` in a Spring Boot app, you're coding against the **JPA specification**; Spring Boot's default underlying implementation providing the actual behavior is **Hibernate**.

4.4 What Hibernate/JPA Solved (and What Still Needed Solving)

- Eliminated manual SQL/ResultSet mapping boilerplate.
- Provided caching, lazy loading, and relationship management (`@OneToMany` , `@ManyToOne`) out of the box.
- **Still needed:** a way to wire together DAOs/Services/Controllers cleanly, manage transactions declaratively, and avoid manually creating a `SessionFactory` / `EntityManagerFactory` in every application — exactly the gap Spring Framework filled next.

5. Introduction to Spring Framework — Why It Was Created

5.1 The Problem Spring Solved (2003)

At the time, "Enterprise Java" meant heavyweight EJB (Enterprise JavaBeans) — requiring a full EJB container, verbose boilerplate interfaces, and painfully slow development/testing cycles. Rod Johnson created Spring as a lightweight alternative built on **plain old Java objects (POJOs)** and **Dependency Injection**, proving you could get enterprise-grade features (transactions, security) without EJB's heaviness.

5.2 Spring's Core Idea

Instead of your code creating and wiring its own dependencies (`new AccountDao()` scattered everywhere), you describe what your objects need, and the **Spring IoC Container** creates and injects them for you — objects become simple, testable POJOs with no framework-specific base classes to extend (unlike EJB).

Before Spring vs With Spring

AccountService manually does: `new AccountRepository()`, `new EmailService()` — tightly coupled, hard to test

— VS —

Spring Container creates `AccountRepository` and `EmailService`, injects them into `AccountService` automatically

5.3 What Spring Actually Provides (High Level)

Module	Purpose
Spring Core (IoC/DI)	Object creation and wiring (Section 12-16)
Spring AOP	Cross-cutting concerns like logging/transactions without cluttering business code (Section 20)
Spring MVC	Web framework built on Servlets, with clean separation of concerns (Section 21+)
Spring Data	Simplifies JPA/Hibernate repository code even further
Spring Security	Authentication/authorization framework
Spring Boot	Auto-configuration and convention-over-configuration on top of all the above (Section 7)

6. Introduction to Spring MVC

6.1 What Spring MVC Added on Top of Spring Core

Spring MVC is Spring's own web framework — built on top of the Servlet API, providing a clean Model-View-Controller structure, annotation-based routing (`@RequestMapping`), and automatic data binding/validation, replacing the need for older frameworks like Struts.

6.2 A Quick Preview (Full Detail in Section 21+)

```
@Controller
public class AccountController {
    @Autowired
    private AccountService accountService; // Spring injects this automatically – no "new"

    @GetMapping("/account/{id}")
    public String getAccount(@PathVariable Long id, Model model) {
        model.addAttribute("account", accountService.findById(id));
        return "accountView"; // name of a view (JSP/Thymeleaf template) to render
    }
}
```

Notice how dramatically cleaner this is than the raw Servlet in Section 2.2 — no manual HTML printing, no manual object creation, clean separation between routing (`@GetMapping`), business logic (`accountService`), and rendering (the returned view name).

7. Introduction to Spring Boot

7.1 The Problem Spring Boot Solved (2014)

Plain Spring (even Spring MVC) still required substantial manual configuration — XML or Java config classes wiring up a DataSource, a ViewResolver, a TransactionManager, and deploying a WAR file to an external Tomcat server you had to install and configure yourself. Spring Boot was created to eliminate nearly all of this via **auto-configuration** and **embedded servers**.

7.2 What Spring Boot Actually Adds

Feature	What It Means
Auto Configuration	Spring Boot inspects what's on your classpath and automatically configures beans you'd otherwise wire manually (Section 33)
Starter Dependencies	One dependency (e.g., <code>spring-boot-starter-web</code>) pulls in everything needed for a typical use case (Section 34)
Embedded Server	Tomcat/Jetty/Undertow is embedded directly in your JAR — just run <code>java -jar app.jar</code> , no separate server install (Section 38)
Opinionated Defaults	Sensible defaults for everything, all overridable via <code>application.yml</code> (Section 35)
Production-Ready Features	Actuator gives health checks/metrics out of the box (Section 37)

7.3 A Complete, Runnable Spring Boot App (Minimal)

```
@SpringBootApplication
public class BankApplication {
    public static void main(String[] args) {
        SpringApplication.run(BankApplication.class, args);    // starts embedded Tomcat + entire Spring context
    }
}
```

This is the entire bootstrap needed — no `web.xml`, no external server install, no manual DataSource/ViewResolver XML configuration for typical use cases. This is precisely why Spring Boot became the default choice for virtually all new Java web/microservices development.

8. Struts vs Spring vs Spring MVC vs Spring Boot — Full Comparison

Aspect	Struts	Spring (Core)	Spring MVC	Spring Boot
Category	Web MVC framework	General-purpose IoC/DI framework	Web MVC framework (built on Spring Core)	Convention-over-configuration layer on top of the entire Spring ecosystem
Configuration	Heavy XML (struts-config.xml)	XML or Java Config, still manual wiring	Annotations + some Java Config, still needs manual bean setup	Auto-configuration — minimal explicit config needed
Server	Deployed as WAR to external Tomcat	N/A (not web-specific)	Deployed as WAR to external Tomcat	Embedded Tomcat — runs as executable JAR
Testability	Harder — tightly coupled to framework base classes	Good — POJOs, easy to unit test	Good	Excellent — plus built-in testing support (Section 45+)
Status today	Largely legacy/retired (Apache Struts 1 esp.)	Still the foundation underneath Spring MVC/Boot	Still used, usually via Spring Boot now	The de facto standard for new Java web/microservices projects

Interview one-liner: "Spring Core provides the IoC container; Spring MVC is a web framework built on top of Spring Core; Spring Boot is a convention-over-configuration layer built on top of both, adding auto-configuration and embedded servers — they're layered, not competing alternatives."

9. Project Structure — Plain Servlet/JSP Application

```
bank-servlet-app/
├── src/main/java/com/bank/
│   ├── servlet/
│   │   ├── AccountServlet.java      # extends HttpServlet, handles /account URL manually
│   │   └── LoginServlet.java
│   ├── dao/
│   │   └── AccountDao.java          # raw JDBC code (Section 3)
│   └── model/
│       └── Account.java             # plain POJO, no annotations
├── src/main/webapp/
│   ├── WEB-INF/
│   │   ├── web.xml                 # manual servlet-to-URL mapping (no annotations)
│   │   └── jsp/
│   │       └── accountView.jsp      # JSP mixing HTML and Java scriptlets
│   └── index.html
└── pom.xml                         # packaging: war — deployed to an external Tomcat
```

Notice everything is manual: URL routing (web.xml), object creation (`new AccountDao()` inside servlets), and no clear separation between the DAO/service/controller layers — this is the exact structure that motivated Struts and later Spring MVC.

10. Project Structure — Spring MVC Application

```

bank-springmvc-app/
├── src/main/java/com/bank/
│   ├── config/
│   │   ├── WebConfig.java           # @EnableWebMvc, ViewResolver bean, interceptor registration
│   │   └── AppConfig.java          # DataSource, TransactionManager beans (Java Config)
│   ├── controller/
│   │   └── AccountController.java    # @Controller, @RequestMapping methods
│   ├── service/
│   │   ├── AccountService.java      # interface
│   │   └── impl/AccountServiceImpl.java # @Service, business logic
│   ├── repository/
│   │   └── AccountRepository.java    # @Repository, Hibernate/JPA-based
│   └── model/
│       └── Account.java              # @Entity
├── src/main/webapp/
│   ├── WEB-INF/
│   │   ├── web.xml                 # registers Spring's DispatcherServlet (Section 22)
│   │   └── views/
│   │       └── accountView.jsp
└── pom.xml                         # packaging: war – still deployed to an external Tomcat

```

Real separation of concerns now exists (controller → service → repository), and Spring's IoC container wires everything via `@Autowired` — but you still write `web.xml` (or a `WebApplicationInitializer`) to register the `DispatcherServlet`, and deploy a WAR to an external, separately-installed Tomcat server.

11. Project Structure — Spring Boot Application (Real Organization Layout)

11.1 Standard Layered Spring Boot Project (Real Org Pattern)

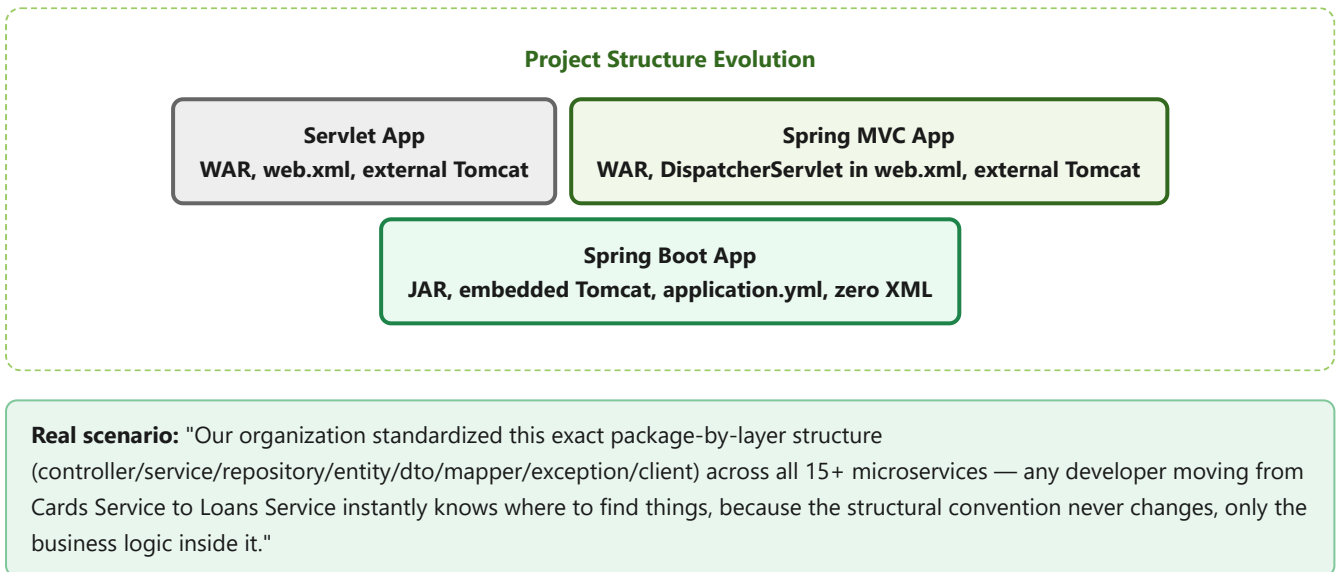
```
accounts-service/
├── src/main/java/com/bank/accounts/
│   ├── AccountServiceApplication.java    # @SpringBootApplication – single entry point, embeds Tomcat
│   ├── config/
│   │   ├── SecurityConfig.java          # Spring Security / JWT configuration
│   │   ├── SwaggerConfig.java           # OpenAPI configuration
│   │   ├── RedisConfig.java             # Cache configuration (Section 52)
│   │   └── CorsConfig.java
│   ├── controller/
│   │   └── AccountController.java        # @RestController – REST API layer
│   ├── service/
│   │   ├── AccountService.java           # interface – business logic contract
│   │   └── impl/AccountServiceImpl.java  # @Service – actual business logic
│   ├── repository/
│   │   └── AccountRepository.java         # extends JpaRepository – Spring Data JPA
│   ├── entity/
│   │   └── Account.java                  # @Entity – JPA-managed database model
│   ├── dto/
│   │   ├── AccountDto.java               # response shape
│   │   └── CreateAccountRequest.java     # request shape
│   ├── mapper/
│   │   └── AccountMapper.java             # Entity ↔ DTO conversion (often MapStruct-generated)
│   ├── exception/
│   │   ├── AccountNotFoundException.java
│   │   └── GlobalExceptionHandler.java   # @RestControllerAdvice
│   └── client/
│       └── CardsFeignClient.java          # inter-service communication
├── src/main/resources/
│   ├── application.yml                   # default config
│   ├── application-dev.yml
│   ├── application-qa.yml
│   ├── application-prod.yml
│   └── db/migration/                     # Flyway/Liquibase SQL migration scripts
│       └── V1__create_account_table.sql
├── src/test/java/com/bank/accounts/
│   ├── controller/AccountControllerTest.java # @WebMvcTest (Section 47)
│   ├── service/AccountServiceTest.java      # Mockito unit tests (Section 46)
│   └── integration/AccountIntegrationTest.java # @SpringBootTest + Testcontainers (Section 49)
├── Dockerfile
├── pom.xml                               # packaging: jar – runs standalone with embedded Tomcat
└── docker-compose.yml                    # local dev: this service + Postgres + Redis
```

11.2 Why This Layered Structure (Real Reasoning)

Layer	Responsibility	Should NOT Contain
Controller	HTTP concerns only — request/response mapping, validation trigger, calling the service	Business logic, direct database access

Service	Business logic, orchestration, transaction boundaries	HTTP-specific code (HttpServletRequest, ResponseEntity)
Repository	Data access only	Business rules/validation
Entity	Database-mapped model	Ever being returned directly from a Controller (Section 45, API guide)
DTO	Public API contract shape	JPA annotations, database concerns

11.3 Struts vs Spring MVC vs Spring Boot Project Structure — Side by Side

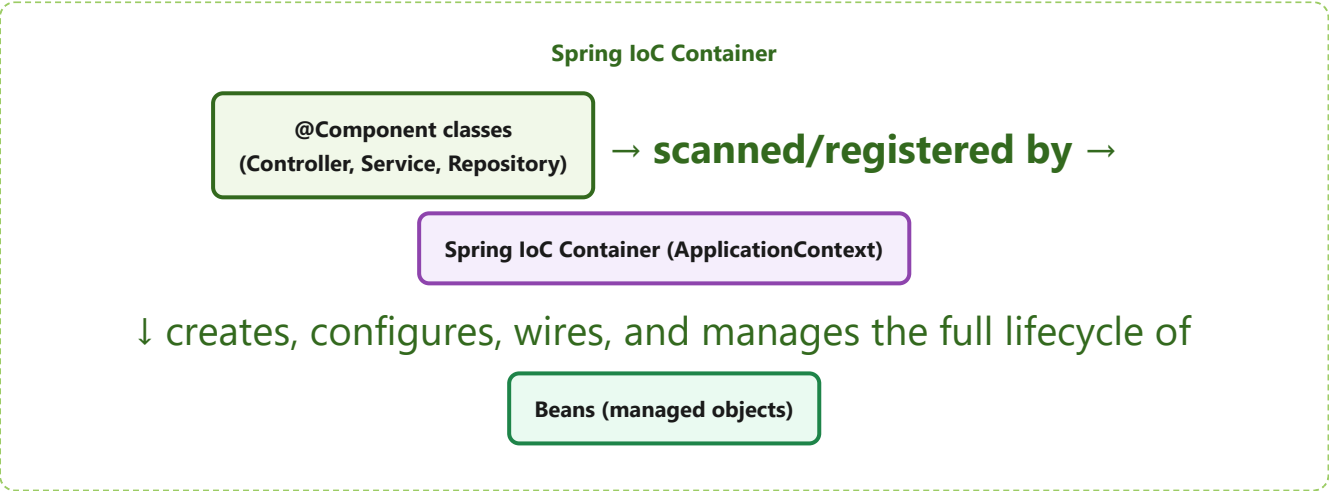


12. Spring Core — The IOC Container

12.1 What Is Inversion of Control (IoC)?

Normally, your code controls object creation (`new AccountService()`). With IoC, that control is "inverted" — a container creates and manages your objects (called **Beans**) for you, and hands them to whoever needs them. You describe dependencies; the container resolves and provides them.

12.2 The IoC Container Visualized



12.3 BeanFactory vs ApplicationContext

	BeanFactory	ApplicationContext
What it is	The most basic IoC container interface — lazy bean initialization	A more feature-rich extension of BeanFactory — the one actually used in real applications
Extra features	—	Event publishing, internationalization, AOP integration, eager singleton initialization by default
Real usage	Rarely used directly	Always what Spring Boot uses under the hood (<code>SpringApplication.run()</code> returns one)

12.4 How the Container Discovers Beans

```
@SpringBootApplication // includes @ComponentScan – scans the current package and sub-packages for
@Component-annotated classes
public class BankApplication { ... }

@Service // a specialization of @Component – registered as a bean automatically
public class AccountService { ... }
```

13. Dependency Injection — Types and Real Examples

13.1 The Three Types of Dependency Injection

Type	How	Recommended?
Constructor Injection	Dependencies passed via the constructor	Yes — the recommended default for mandatory dependencies
Setter Injection	Dependencies set via a setter method	Used for optional dependencies
Field Injection	<code>@Autowired</code> directly on a field	Discouraged in production code — harder to unit test, hides required dependencies

13.2 Real Code — All Three Types

```
// Constructor Injection (RECOMMENDED) – dependencies are final, impossible to construct in an invalid state
@Service
public class AccountService {
    private final AccountRepository accountRepository;
    private final EmailService emailService;

    public AccountService(AccountRepository accountRepository, EmailService emailService) {
        this.accountRepository = accountRepository;
        this.emailService = emailService;
    }
}

// With Lombok, even shorter:
@Service
@RequiredArgsConstructor // generates the constructor automatically for all final fields
public class AccountService {
    private final AccountRepository accountRepository;
    private final EmailService emailService;
}

// Setter Injection
@Service
public class ReportService {
    private NotificationService notificationService; // optional dependency

    @Autowired(required = false)
    public void setNotificationService(NotificationService notificationService) {
        this.notificationService = notificationService;
    }
}

// Field Injection (DISCOURAGED)
@Service
public class LegacyService {
    @Autowired
    private AccountRepository accountRepository; // works, but can't be made final, hard to unit test without reflection
}
```

Interview tip: Constructor injection is preferred because it makes dependencies explicit and immutable (`final`), fails fast at startup if a dependency is missing, and makes unit testing trivial — you just call `new AccountService(mockRepo, mockEmailService)` without needing Spring or reflection tricks at all.

13.3 Resolving Ambiguity — @Qualifier and @Primary

```
public interface NotificationSender { void send(String message); }

@Service("smsSender")
public class SmsSender implements NotificationSender { ... }

@Service("emailSender")
@Primary // used by default when multiple candidates exist and no @Qualifier is specified
public class EmailSender implements NotificationSender { ... }

@Service
public class AlertService {
    private final NotificationSender sender;

    public AlertService(@Qualifier("smsSender") NotificationSender sender) { // explicitly picks the SMS bean
        this.sender = sender;
    }
}
```

14. Spring Bean Lifecycle

14.1 The Full Bean Lifecycle, Step by Step



14.2 Real Code — Hooking Into the Lifecycle

```
@Component
public class ConnectionPoolManager {

    @PostConstruct
    public void init() {
        log.info("Initializing connection pool..."); // runs once, right after dependency injection completes
    }

    @PreDestroy
    public void cleanup() {
        log.info("Closing connection pool gracefully..."); // runs during graceful application shutdown
    }
}
```

Real scenario: "Our Kafka consumer service uses `@PreDestroy` to finish processing any in-flight message and commit its offset before the pod actually terminates during a Kubernetes rolling deployment — without this, a mid-shutdown pod kill could silently drop a message."

15. Spring Bean Scopes

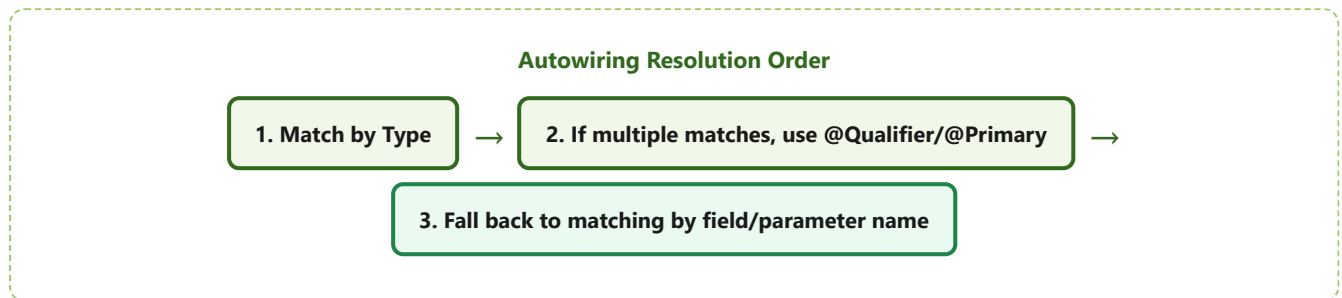
Scope	Meaning	Real Use Case
singleton (default)	Exactly one instance per Spring container, shared everywhere	Stateless services, repositories — the vast majority of beans
prototype	A brand-new instance created every time the bean is requested	Stateful, non-thread-safe helper objects needed per-use
request	One instance per HTTP request (web-aware contexts only)	Request-scoped data holders in a web app
session	One instance per HTTP session	User-session-specific state (rare in stateless REST APIs)
application	One instance per ServletContext	Rarely used directly

```
@Component
@Scope("prototype")
public class ReportGenerator { ... } // a new instance every time it's injected/requested
```

Common mistake: Injecting a `prototype` -scoped bean into a `singleton` -scoped bean via normal field/constructor injection only creates the prototype bean **once** (at singleton creation time) — you need a special pattern (`ObjectProvider` or a scoped proxy) to actually get a fresh prototype instance on every use.

16. Autowiring in Spring

16.1 How Autowiring Resolves Dependencies



16.2 Real Autowiring Examples

```
@Autowired // matches by type – works fine if only ONE bean of type NotificationSender exists
private NotificationSender sender;

@Autowired
@Qualifier("smsSender") // disambiguates when multiple beans of the same type exist
private NotificationSender sender;

@Autowired(required = false) // won't throw an error if no matching bean is found – sender stays null
private NotificationSender optionalSender;
```

Real production code almost always uses constructor injection (Section 13.2) instead of field-level `@Autowired` — but understanding autowiring resolution rules is still a very common interview question.

17. Configuration — XML vs Java Config vs Annotations

17.1 The Three Ways to Configure Spring (Historical Evolution)

Style	Era	Still Used Today?
XML Configuration	Original Spring style (2003+)	Rare in new projects, still seen in legacy codebases
Java Config (<code>@Configuration</code> / <code>@Bean</code>)	Introduced Spring 3 (2009)	Common for third-party beans you don't own the source of
Annotation-Based (<code>@Component</code> , <code>@Service</code> , etc.)	Increasingly dominant since Spring 2.5+/Spring Boot	The default approach in virtually all modern Spring Boot code

17.2 Real XML Configuration (Legacy Style)

```
<beans xmlns="http://www.springframework.org/schema/beans">
  <bean id="accountService" class="com.bank.service.AccountService">
    <constructor-arg ref="accountRepository"/>
  </bean>
  <bean id="accountRepository" class="com.bank.repository.AccountRepositoryImpl"/>
</beans>
```

17.3 Real Java Config

```
@Configuration
public class AppConfig {
    @Bean
    public AccountService accountService(AccountRepository repo) {
        return new AccountService(repo); // used typically for beans you don't control the source of (3rd-party libraries)
    }

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}
```

17.4 Real Annotation-Based Configuration (The Modern Default)

```
@Service
@RequiredArgsConstructor
public class AccountService {
    private final AccountRepository accountRepository; // Spring finds and injects this automatically, zero explicit config needed
}
```

Real practice: Use annotations (`@Component` / `@Service` / `@Repository`) for your own application classes, and Java Config (`@Configuration` / `@Bean`) specifically for wiring third-party library objects (like a `RestTemplate` , `ObjectMapper` , or a Redis `RedisTemplate`) that you can't annotate directly since you don't own their source code.

18. Spring Annotations — The Complete Reference

Annotation	Purpose
<code>@Component</code>	Generic stereotype — marks a class as a Spring-managed bean
<code>@Service</code>	Specialization of <code>@Component</code> for the service/business logic layer
<code>@Repository</code>	Specialization of <code>@Component</code> for the data access layer; also enables automatic exception translation (JDBC exceptions → Spring's <code>DataAccessException</code> hierarchy)
<code>@Controller</code>	Specialization of <code>@Component</code> for MVC web controllers returning views
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code> combined — every method return value is serialized directly as the HTTP response body (JSON)
<code>@Configuration</code>	Marks a class as a source of bean definitions (Java Config)
<code>@Bean</code>	Marks a method inside a <code>@Configuration</code> class as producing a bean
<code>@Autowired</code>	Requests dependency injection for a field/constructor/setter
<code>@Qualifier</code>	Disambiguates which specific bean to inject when multiple candidates of the same type exist
<code>@Primary</code>	Marks a bean as the default choice when multiple candidates exist and no <code>@Qualifier</code> is specified
<code>@Value</code>	Injects a value from <code>application.yml/properties</code> (or a SpEL expression) into a field
<code>@Scope</code>	Declares a bean's scope (singleton, prototype, etc.)
<code>@PostConstruct</code> / <code>@PreDestroy</code>	Lifecycle hooks (Section 14)
<code>@ComponentScan</code>	Tells Spring which packages to scan for <code>@Component</code> -annotated classes
<code>@Transactional</code>	Declares a method/class as running within a database transaction (Section 41)
<code>@Aspect</code>	Marks a class as an AOP aspect (Section 19)
<code>@SpringBootApplication</code>	Combines <code>@Configuration</code> + <code>@EnableAutoConfiguration</code> + <code>@ComponentScan</code> (Section 33)

19. Spring Expression Language (SpEL)

19.1 What SpEL Is

SpEL (Spring Expression Language) is a powerful expression language that lets you evaluate expressions dynamically inside annotations and configuration — used heavily in `@Value`, Spring Security method security, and caching annotations.

19.2 Real SpEL Examples

```
// Injecting a simple property value
@Value("${app.max-retry-count}")
private int maxRetryCount;

// Injecting a computed/default value with SpEL
@Value("#{systemProperties['user.timezone'] ?: 'UTC'}")
private String timezone;

// Referencing another bean's property via SpEL
@Value("#{accountConfig.defaultCurrency}")
private String defaultCurrency;

// SpEL used in Spring Security method-level authorization
@PreAuthorize("#customerId == authentication.principal.customerId or hasRole('ADMIN')")
public AccountDto getAccount(Long customerId) { ... }

// SpEL used in caching to build a dynamic cache key (Section 51)
@Cacheable(value = "accounts", key = "#customerId + '-' + #accountType")
public List<Account> findAccounts(Long customerId, String accountType) { ... }
```

Real scenario: "Our `@PreAuthorize` SpEL expressions enforce that a customer can only view their own account data unless they hold an ADMIN role — this security rule lives declaratively right next to the method it protects, instead of being buried in imperative if-checks scattered through the service layer."

20. AOP — Aspect Oriented Programming

20.1 What Problem AOP Solves

Cross-cutting concerns (logging, security checks, transaction management, auditing) would otherwise need to be repeated in every single business method. AOP lets you define this logic **once**, as an "Aspect," and apply it declaratively across many methods/classes matching a pattern ("Pointcut") — without cluttering the actual business logic.

20.2 Core AOP Terminology

Term	Meaning
Aspect	A module encapsulating a cross-cutting concern (e.g., a LoggingAspect class)
Advice	The actual action taken (what code runs) — Before, After, Around, AfterReturning, AfterThrowing
Pointcut	An expression defining WHICH methods the advice applies to
Join Point	A specific point during execution (e.g., a method call) where an aspect can be applied
Weaving	The process of applying aspects to target objects (Spring does this at runtime via proxies)

20.3 Real AOP Code — Logging Every Service Method's Execution Time

```
@Aspect
@Component
public class PerformanceLoggingAspect {

    @Around("execution(* com.bank.accounts.service.*(..))") // Pointcut: every method in the service
    package
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed(); // actually calls the real method
        long duration = System.currentTimeMillis() - start;
        log.info("{} executed in {}ms", joinPoint.getSignature(), duration);
        return result;
    }
}
```

20.4 Real AOP Code — Custom @Auditable Annotation (Recap From API Guide)

```
@Aspect
@Component
public class AuditAspect {
    @Around("@annotation(Auditable)")
    public Object audit(ProceedingJoinPoint joinPoint) throws Throwable {
        String user = SecurityContextHolder.getContext().getAuthentication().getName();
        Object result = joinPoint.proceed();
        auditLogRepository.save(new AuditLog(user, joinPoint.getSignature().getName(), Instant.now()));
        return result;
    }
}
```

20.5 How AOP Actually Works Under the Hood

Spring AOP creates a **proxy** object wrapping your real bean — when you call a method, you're actually calling the proxy first, which runs the aspect logic and then delegates to the real method. This is why `@Transactional` /AOP-based advice doesn't work when calling a method on `this` from within the same class — that internal call bypasses the proxy entirely.

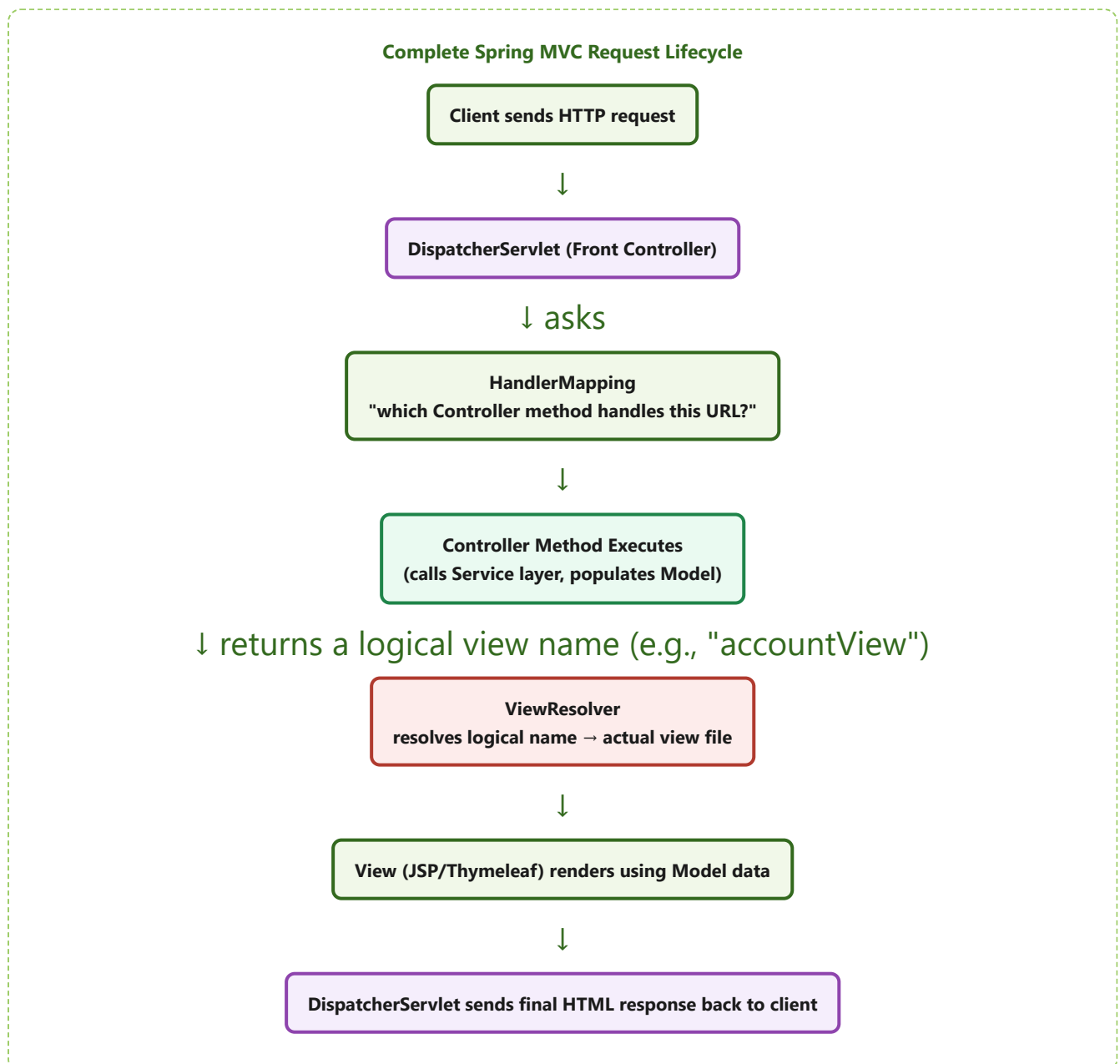
Common gotcha: If method A calls method B in the same class, and B has `@Transactional`, calling A won't trigger B's transaction behavior — because the call goes directly (`this.B()`), never passing through the Spring-generated proxy. Real fix: move B into a separate bean/service and inject it, or use `AopContext.currentProxy()` (less clean).

21. Spring MVC Architecture — The Full Picture

21.1 The MVC Pattern

Model-View-Controller separates an application into three responsibilities: the **Model** (data), the **View** (presentation/rendering), and the **Controller** (handles requests, coordinates between Model and View) — this separation is exactly what raw Servlets (Section 2) lacked.

21.2 Full Spring MVC Request Flow Diagram



This entire flow (Section 21.2) is one of the single most commonly asked Spring interview questions — "explain the Spring MVC request flow end to end" — make sure you can draw and narrate this diagram confidently.

22. DispatcherServlet Explained in Depth

22.1 What Is DispatcherServlet?

DispatcherServlet is Spring MVC's **Front Controller** — a single Servlet that receives every incoming HTTP request for the application and delegates to the appropriate Controller, ViewResolver, and other components. It's the one and only Servlet a Spring MVC/Boot web app actually registers with the underlying Servlet container.

22.2 How DispatcherServlet Gets Registered

```
<!-- Traditional Spring MVC - web.xml -->
<servlet>
  <servlet-name>dispatcher</servlet-name>
  <servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>dispatcher</servlet-name>
  <url-pattern>/</url-pattern> <!-- catches EVERY request -->
</servlet-mapping>

// Spring Boot - registered AUTOMATICALLY, zero configuration needed
// (auto-configured by DispatcherServletAutoConfiguration when spring-boot-starter-web is on the classpath)
```

22.3 What DispatcherServlet Actually Coordinates

Component	Role
HandlerMapping	Determines which controller method should handle the incoming URL
HandlerAdapter	Actually invokes the chosen controller method with the right arguments
HandlerExceptionResolver	Handles exceptions thrown during request processing (Section 29)
ViewResolver	Resolves a returned logical view name into an actual renderable View (Section 24)
HandlerInterceptor	Pre/post-processing hooks around controller execution (Section 30)

Interview tip: DispatcherServlet is a real implementation of the "Front Controller" design pattern — a single entry point that all requests pass through, rather than each URL being handled by a completely separate, independent Servlet as in the raw Servlet era (Section 2).

23. Controllers in Spring MVC

23.1 @Controller vs @RestController

	@Controller	@RestController
Return value meaning	A logical view name to be resolved and rendered (HTML page)	The actual response body directly (typically JSON), via implicit @ResponseBody on every method
Typical use	Traditional server-rendered web apps (JSP/Thymeleaf)	REST APIs — the standard choice for Spring Boot microservices

23.2 Real @Controller (Server-Rendered View)

```
@Controller
public class AccountViewController {
    @GetMapping("/accounts/{id}/view")
    public String viewAccount(@PathVariable Long id, Model model) {
        model.addAttribute("account", accountService.findById(id));
        return "accountView"; // a VIEW NAME, resolved by ViewResolver into an actual JSP/Thymeleaf file
    }
}
```

23.3 Real @RestController (JSON API)

```
@RestController
@RequestMapping("/api/v1/accounts")
public class AccountController {
    @GetMapping("/{id}")
    public AccountDto getAccount(@PathVariable Long id) {
        return accountService.findById(id); // serialized DIRECTLY to JSON, no view resolution happens at all
    }
}
```

24. View Resolver

24.1 What ViewResolver Does

A ViewResolver translates the logical view name a controller returns (e.g., `"accountView"`) into an actual View object capable of rendering — typically by prepending a prefix and appending a suffix to form a real file path.

24.2 Real ViewResolver Configuration

```
@Bean
public InternalResourceViewResolver viewResolver() {
    InternalResourceViewResolver resolver = new InternalResourceViewResolver();
    resolver.setPrefix("/WEB-INF/views/");
    resolver.setSuffix(".jsp");
    return resolver;    // "accountView" resolves to /WEB-INF/views/accountView.jsp
}

# Spring Boot equivalent – Thymeleaf auto-configuration via application.yml
spring:
  thymeleaf:
    prefix: classpath:/templates/
    suffix: .html    # "accountView" resolves to classpath:/templates/accountView.html
```

Real-world note: Most modern Spring Boot REST APIs never use a ViewResolver at all — `@RestController` bypasses view resolution entirely by serializing return values directly to JSON. ViewResolvers are only relevant for traditional server-rendered UIs (JSP/Thymeleaf), covered further in Sections 31-32.

25. Request Mapping — Full Reference

25.1 The Full @RequestMapping Family

Annotation	Equivalent To
<code>@GetMapping</code>	<code>@RequestMapping(method = RequestMethod.GET)</code>
<code>@PostMapping</code>	<code>@RequestMapping(method = RequestMethod.POST)</code>
<code>@PutMapping</code>	<code>@RequestMapping(method = RequestMethod.PUT)</code>
<code>@PatchMapping</code>	<code>@RequestMapping(method = RequestMethod.PATCH)</code>
<code>@DeleteMapping</code>	<code>@RequestMapping(method = RequestMethod.DELETE)</code>

25.2 Real Parameter-Binding Annotations

```
@RestController
@RequestMapping("/api/v1/accounts")
public class AccountController {

    @GetMapping("/{id}")    // path variable
    public AccountDto getAccount(@PathVariable Long id) { ... }

    @GetMapping            // query parameters
    public List<AccountDto> search(@RequestParam String status,
                                   @RequestParam(defaultValue = "0") int page) { ... }

    @PostMapping           // request body
    public AccountDto create(@RequestBody @Valid CreateAccountRequest request) { ... }

    @GetMapping("/lookup") // request header
    public AccountDto lookup(@RequestHeader("X-Customer-Id") String customerId) { ... }

    @PostMapping("/upload") // multipart file upload
    public void upload(@RequestPart MultipartFile file) { ... }
}
```

26. Model and ModelMap

26.1 What Model/ModelMap Are For

In traditional (non-REST) Spring MVC controllers, the `Model` (or `ModelMap`) is how the controller passes data to the view for rendering — attributes added to it become available as variables inside the JSP/Thymeleaf template.

26.2 Real Code

```
@Controller
public class AccountViewController {
    @GetMapping("/accounts/{id}/view")
    public String viewAccount(@PathVariable Long id, Model model) {
        Account account = accountService.findById(id);
        model.addAttribute("account", account);           // available as ${account} in the view
        model.addAttribute("pageTitle", "Account Details");
        return "accountView";
    }
}
```

```
<!-- accountView.jsp -->
<h1>${pageTitle}</h1>
<p>Balance: ${account.balance}</p>
```

	Model	ModelMap	ModelAndView
Type	Interface	A concrete Map-like implementation	Combines both the model data AND the view name in one object
Common use	Most common, simplest choice	When Map-specific methods are convenient	When you want to set the view name and model together in one return object

27. Form Handling in Spring MVC

27.1 Real Form-Backed Controller

```
@Controller
@RequestMapping("/accounts")
public class AccountFormController {

    @GetMapping("/new")
    public String showForm(Model model) {
        model.addAttribute("accountForm", new CreateAccountForm()); // empty form-backing object
        return "newAccountForm";
    }

    @PostMapping("/new")
    public String submitForm(@ModelAttribute @Valid CreateAccountForm form,
                           BindingResult bindingResult) {
        if (bindingResult.hasErrors()) {
            return "newAccountForm"; // re-render the SAME form with validation error messages
        }
        accountService.create(form);
        return "redirect:/accounts/success"; // Post-Redirect-Get pattern – avoids duplicate submission on
refresh
    }
}
```

27.2 Real JSP Form Tags

```
<form:form modelAttribute="accountForm" method="POST">
    <form:label path="customerName">Customer Name</form:label>
    <form:input path="customerName"/>
    <form:errors path="customerName" cssClass="error"/>

    <form:label path="initialDeposit">Initial Deposit</form:label>
    <form:input path="initialDeposit"/>
    <button type="submit">Create Account</button>
</form:form>
```

Post-Redirect-Get (PRG) pattern: After a successful form POST, redirect to a GET URL rather than rendering a view directly — this prevents the classic "confirm form resubmission" browser warning if the user refreshes the page after submitting.

28. Data Binding

28.1 What Data Binding Does

Data Binding is Spring's automatic process of converting incoming HTTP request data (form fields, query params, JSON body) into strongly-typed Java objects — you never manually call `request.getParameter("name")` and parse it yourself.

28.2 Real Example — Automatic Type Conversion

```
// Incoming form data: customerName=John&initialDeposit=5000.50&birthDate=1990-05-15
public class CreateAccountForm {
    private String customerName;
    private BigDecimal initialDeposit;    // Spring automatically converts "5000.50" → BigDecimal

    @DateTimeFormat(pattern = "yyyy-MM-dd")
    private LocalDate birthDate;          // Spring automatically converts "1990-05-15" → LocalDate
}
```

28.3 Custom Property Editors / Converters (For Non-Standard Types)

```
@Component
public class StringToAccountTypeConverter implements Converter<String, AccountType> {
    @Override
    public AccountType convert(String source) {
        return AccountType.valueOf(source.toUpperCase());    // custom conversion logic for a domain enum
    }
}
```

29. Validation in Spring MVC

29.1 Recap — Bean Validation Annotations (Full Detail in API Guide Section 15)

```
public class CreateAccountForm {
    @NotBlank(message = "Customer name is required")
    private String customerName;

    @NotNull @Positive(message = "Deposit must be positive")
    private BigDecimal initialDeposit;
}
```

29.2 Real Validation in a Form-Based (View-Rendering) Controller

```
@PostMapping("/new")
public String submitForm(@ModelAttribute @Valid CreateAccountForm form, BindingResult bindingResult) {
    if (bindingResult.hasErrors()) {
        return "newAccountForm"; // BindingResult MUST immediately follow the @Valid parameter, or Spring
        // throws an exception instead
    }
    ...
}
```

Common gotcha: In a `@Controller` (view-based) method, `BindingResult` must appear as the parameter immediately after the `@Valid`-annotated object — if anything else comes between them, or `BindingResult` is omitted, Spring throws a `MethodArgumentNotValidException` instead of letting you handle errors gracefully in the same method.

29.3 Custom Validators (Recap From API Guide, Section 15.3)

For business-rule validation beyond simple annotations (e.g., "account type must be one of these specific values"), implement a custom `ConstraintValidator` — already covered in full in the API Development guide.

30. Exception Handling in Spring MVC

30.1 Three Ways to Handle Exceptions

Approach	Scope	Real Use
<code>@ExceptionHandler</code> (local)	Only within the same controller class	A specific exception unique to one controller
<code>@ControllerAdvice</code> / <code>@RestControllerAdvice</code> (global)	Application-wide, across all controllers	The standard real-world approach (full example already in API guide Section 14)
<code>HandlerExceptionResolver</code> (low-level)	Application-wide, more manual/programmatic control	Rarely needed directly — <code>@ControllerAdvice</code> covers nearly all real cases

30.2 Local Exception Handling — Real Example

```
@Controller
public class AccountViewController {

    @GetMapping("/accounts/{id}/view")
    public String viewAccount(@PathVariable Long id, Model model) {
        return "accountView";
    }

    @ExceptionHandler(AccountNotFoundException.class) // only handles this exception WITHIN this one controller
    public String handleNotFound(AccountNotFoundException ex, Model model) {
        model.addAttribute("errorMessage", ex.getMessage());
        return "errorPage";
    }
}
```

Full production-grade global exception handling with a consistent JSON error response shape is already covered in depth in the API Development guide (Section 14) — the concept and annotations are identical whether you're building a REST API or a traditional server-rendered app; only the response TYPE (JSON vs a rendered error view) differs.

31. Interceptors

31.1 What Interceptors Do

A **HandlerInterceptor** lets you run code before, after, or around controller method execution — similar in spirit to AOP (Section 20) but specifically integrated into the Spring MVC request lifecycle, commonly used for logging, authentication checks, and adding common response headers.

31.2 Real Interceptor Implementation

```
@Component
public class RequestTimingInterceptor implements HandlerInterceptor {

    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler) {
        request.setAttribute("startTime", System.currentTimeMillis());
        return true; // returning false stops the request here – the controller method never runs
    }

    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse response, Object handler,
Exception ex) {
        long start = (Long) request.getAttribute("startTime");
        log.info("{} took {}ms", request.getRequestURI(), System.currentTimeMillis() - start);
    }
}

// Registration
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new RequestTimingInterceptor())
            .addPathPatterns("/api/**")
            .excludePathPatterns("/api/v1/auth/**"); // skip auth endpoints
    }
}
```

31.3 Interceptor vs Filter — A Common Interview Question

	Servlet Filter	Spring HandlerInterceptor
Layer	Servlet container level — runs before Spring even gets the request	Spring MVC level — runs within DispatcherServlet's processing
Access to Spring context	No direct access to Spring beans/handler method info	Full access to the Spring container, the specific controller method being called, etc.
Best for	Cross-cutting concerns not specific to Spring (e.g., CORS, gzip compression, raw request logging)	Spring-aware concerns (e.g., checking annotations on the target controller method)

32. JSP Integration & Thymeleaf Integration

32.1 What JSP Is (Recap)

JavaServer Pages (JSP) lets you write HTML pages with embedded Java logic and custom tags, compiled into a Servlet by the container at runtime — the traditional view technology for Spring MVC applications before Thymeleaf became the modern default.

32.2 Real JSP Page Using Spring/JSTL Tags

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<html>
<body>
  <h1>Account Details</h1>
  <p>Balance: ${account.balance}</p>

  <c:if test="${account.status == 'ACTIVE'}">
    <span class="badge-green">Active</span>
  </c:if>

  <c:forEach var="card" items="${account.cards}">
    <li>Card ending in ${card.last4Digits}</li>
  </c:forEach>
</body>
</html>
```

32.3 Real Limitations That Led Away From JSP

- JSP files must be compiled by a Servlet container — can't easily be unit tested standalone or previewed outside a running server.
- Not naturally packagable inside an executable JAR (Spring Boot's embedded Tomcat has restrictions here) — this alone pushed most new Spring Boot projects toward Thymeleaf.
- JSTL tag syntax feels dated compared to modern template engines.

32.4 Thymeleaf Integration (The Modern Spring Boot Default)

Thymeleaf is a modern server-side template engine designed to work naturally with Spring Boot's embedded server (no compilation-in-container restriction like JSP) — templates are valid HTML that can even be opened and previewed directly in a browser without running the application at all ("natural templating").

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<!-- src/main/resources/templates/accountView.html -->
<html xmlns:th="http://www.thymeleaf.org">
<body>
  <h1>Account Details</h1>
  <p th:text="'Balance: ' + ${account.balance}">Balance placeholder</p>

  <span th:if="${account.status == 'ACTIVE'}" class="badge-green">Active</span>

  <ul>
    <li th:each="card : ${account.cards}" th:text="'Card ending in ' + ${card.last4Digits}"></li>
  </ul>
</body>
</html>
```

32.5 JSP vs Thymeleaf — Comparison

Aspect	JSP	Thymeleaf
Compilation	Compiled into a Servlet by the container at runtime	Parsed directly as HTML by the Thymeleaf engine, no container-specific compilation
Works well with Spring Boot embedded JAR	Awkward, has known limitations	Yes — the natural default choice for Spring Boot apps
Natural templating (valid HTML on its own)	No — JSP syntax isn't valid plain HTML	Yes — designers can open the .html file directly in a browser
Current recommendation	Legacy projects only	The default choice for any new Spring Boot server-rendered UI

Interview tip: Most new Spring Boot projects that need server-rendered HTML (rather than a separate React/Angular SPA) use Thymeleaf, not JSP — JSP is mostly relevant now for understanding/maintaining legacy Spring MVC applications.

33. Spring Boot — Auto Configuration Explained

33.1 How Auto Configuration Actually Works

When your application starts, Spring Boot inspects what's on the classpath (which JAR dependencies are present) and conditionally registers beans you would otherwise have to configure manually — e.g., seeing `spring-boot-starter-data-jpa` and a database driver on the classpath, it automatically configures a `DataSource`, an `EntityManagerFactory`, and a `TransactionManager`.

33.2 The Mechanism — @Conditional Annotations

```
// Simplified version of what a real Spring Boot auto-configuration class looks like internally
@Configuration
@ConditionalOnClass(DataSource.class)           // only activates if DataSource is on the classpath
@ConditionalOnMissingBean(DataSource.class)      // only activates if the developer hasn't already defined
their own DataSource bean
@EnableConfigurationProperties(DataSourceProperties.class)
public class DataSourceAutoConfiguration {
    @Bean
    public DataSource dataSource(DataSourceProperties properties) {
        return properties.initializeDataSourceBuilder().build();
    }
}
```

33.3 @SpringBootApplication — What It Really Combines

@SpringBootApplication Is Three Annotations in One

@Configuration
"this class can define beans"

@EnableAutoConfiguration
"turn on auto-configuration"

@ComponentScan
"scan this package + sub-packages for beans"

33.4 Overriding Auto Configuration (You're Always in Control)

```
// Defining your OWN DataSource bean automatically disables Spring Boot's auto-configured one
@Bean
public DataSource dataSource() {
    return DataSourceBuilder.create()
        .url("jdbc:postgresql://custom-host:5432/mydb")
        .build(); // @ConditionalOnMissingBean means auto-config backs off entirely once you provide your own
}
```

Interview tip: Auto-configuration always defers to explicit developer configuration — it's designed to be a sensible default, never a forced behavior, thanks to `@ConditionalOnMissingBean`-style checks throughout Spring Boot's internals.

33.5 Debugging What Got Auto-Configured

```
# Run with this flag to see a full report of what was and wasn't auto-configured, and why
java -jar app.jar --debug

# Or check programmatically via Actuator
GET /actuator/conditions
```

34. Starter Dependencies

34.1 What Starters Are

A "starter" is a curated dependency bundle — one Maven/Gradle dependency that pulls in every library typically needed for a specific use case, with tested, compatible versions, so you don't have to hunt down and manually align a dozen individual library versions yourself.

34.2 Real Starters Commonly Used

Starter	Pulls In
<code>spring-boot-starter-web</code>	Spring MVC, embedded Tomcat, Jackson (JSON), validation basics
<code>spring-boot-starter-data-jpa</code>	Hibernate, Spring Data JPA, HikariCP connection pool
<code>spring-boot-starter-security</code>	Spring Security core + auto-configuration
<code>spring-boot-starter-test</code>	JUnit 5, Mockito, AssertJ, Spring Test, MockMvc — everything for testing (Sections 47-49)
<code>spring-boot-starter-actuator</code>	Production-ready monitoring endpoints (Section 37)
<code>spring-boot-starter-data-redis</code>	Redis client + Spring Data Redis (Section 53)
<code>spring-boot-starter-validation</code>	Bean Validation (Hibernate Validator)
<code>spring-boot-starter-thymeleaf</code>	Thymeleaf template engine integration (Section 32)

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <!-- ONE line brings in Spring MVC + embedded Tomcat + Jackson – no version numbers needed,
        inherited from the spring-boot-starter-parent / dependency management BOM -->
</dependencies>
```

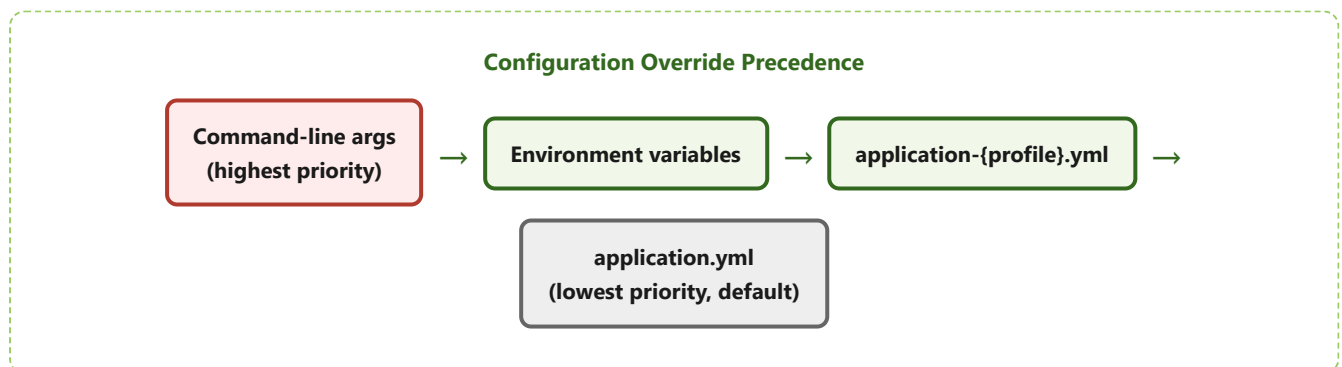
35. Application Properties / YAML Configuration

35.1 Properties vs YAML — Same Purpose, Different Syntax

```
# application.properties
server.port=8081
spring.datasource.url=jdbc:postgresql://localhost:5432/accounts_db
spring.jpa.hibernate.ddl-auto=validate
logging.level.com.bank=INFO

# application.yml — same config, hierarchical/nested (generally preferred in real orgs)
server:
  port: 8081
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/accounts_db
  jpa:
    hibernate:
      ddl-auto: validate
logging:
  level:
    com.bank: INFO
```

35.2 Real Configuration Precedence (Highest to Lowest — Common Interview Q)



```
# Command-line override wins over everything in application.yml
java -jar app.jar --server.port=9090

# Environment variable equivalent (common in Docker/Kubernetes)
SERVER_PORT=9090 java -jar app.jar
```

36. Spring Profiles

36.1 What Profiles Solve

Profiles let the exact same JAR run with different configuration per environment (Dev, QA, Prod) — critical for the "build once, promote everywhere" principle covered in the CI/CD guide (never rebuild the artifact per environment, only change config).

36.2 Real Profile-Specific Files

```
# application.yml – common defaults
spring:
  application:
    name: accounts-service

# application-dev.yml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/accounts_dev
  logging:
    level:
      root: DEBUG

# application-prod.yml
spring:
  datasource:
    url: jdbc:postgresql://prod-db-cluster:5432/accounts_prod
  logging:
    level:
      root: INFO
```

```
# Activating a profile at runtime
java -jar accounts-service.jar --spring.profiles.active=prod
```

36.3 @Profile Annotation on Beans

```
@Configuration
public class NotificationConfig {

    @Bean
    @Profile("prod")
    public NotificationSender realSmsSender() {
        return new TwilioSmsSender(); // real SMS provider only active in production
    }

    @Bean
    @Profile("!prod") // active in every profile EXCEPT prod
    public NotificationSender fakeSmsSender() {
        return new ConsoleLoggingSmsSender(); // just logs to console in dev/QA, avoids real SMS costs
    }
}
```

37. Spring Boot Actuator

Full detail already covered in the Observability & Monitoring guide (Section 20) and the Microservices guide (Section 33) — Actuator exposes production-ready operational endpoints (health, metrics, env, loggers, thread dump, heap dump) that form the backbone of Kubernetes probes, Prometheus scraping, and live production troubleshooting.

Endpoint	Purpose
<code>/actuator/health</code>	UP/DOWN status, including DB/Redis/Kafka connectivity checks
<code>/actuator/info</code>	Build/version metadata
<code>/actuator/metrics</code>	Micrometer-collected metrics
<code>/actuator/env</code>	All active configuration properties (secure this in prod!)
<code>/actuator/beans</code>	Every bean currently registered in the ApplicationContext — great for debugging DI issues
<code>/actuator/mappings</code>	Every registered @RequestMapping URL in the application

```
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics, prometheus, beans, mappings
  endpoint:
    health:
      show-details: when-authorized # don't leak internal DB/dependency details to unauthenticated callers
```

38. Embedded Tomcat

38.1 What "Embedded" Actually Means

Instead of installing Tomcat separately on a server and deploying a WAR file into it, Spring Boot packages Tomcat's core libraries directly **inside** your application's executable JAR — `java -jar app.jar` starts both your application code AND a fully functional Tomcat server in one process.

38.2 Real Benefits

- No separate server installation/version management across environments.
- The exact same Tomcat version is guaranteed in Dev, QA, and Prod (bundled with the JAR) — eliminates "works on my Tomcat but not theirs" issues.
- Fits perfectly with Docker/Kubernetes — one container = one self-contained runnable JAR, no separate app-server layer needed.

38.3 Switching to a Different Embedded Server

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <exclusions>
    <exclusion>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-tomcat</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-undertow</artifactId>  <!-- lighter-weight alternative to Tomcat -->
</dependency>
```

Embedded Server	Notes
Tomcat (default)	Most widely used, mature, well understood, thread-per-request model
Jetty	Lightweight, historically popular for embedded use cases
Undertow	High-performance, non-blocking, commonly paired with Spring WebFlux (reactive)

39. CommandLineRunner & ApplicationRunner

39.1 What They're For

Both let you run custom code exactly once, right after the Spring application context is fully initialized but before the application starts actually serving traffic — commonly used for startup data seeding, cache warm-up, or verifying critical dependencies are reachable.

39.2 Real Code — Both Variants

```
@Component
public class DataSeeder implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        if (accountRepository.count() == 0) {
            accountRepository.save(new Account("Demo Customer", BigDecimal.valueOf(1000))); # seed initial
demo data in dev/QA only
        }
    }
}

@Component
public class ApplicationRunnerExample implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) throws Exception {
        // ApplicationArguments gives structured access to named options, e.g. --env=prod
        if (args.containsOption("env")) {
            log.info("Started with env={}", args.getOptionValues("env"));
        }
    }
}
```

	CommandLineRunner	ApplicationRunner
Argument access	Raw <code>String[] args</code>	Structured <code>ApplicationArguments</code> (named options vs positional args)
Real use	Simple startup tasks	When you need to distinguish <code>--key=value</code> options cleanly

Real practice: Never use `CommandLineRunner` / `ApplicationRunner` for production data seeding — use a proper database migration tool (Flyway/Liquibase, Section 11.1) so seed data is versioned, repeatable, and doesn't accidentally run destructive logic on every single application restart.

40. @ConfigurationProperties

40.1 @Value vs @ConfigurationProperties

For a handful of individual properties, `@Value` (Section 19) works fine. For a group of related settings, `@ConfigurationProperties` binds an entire configuration section to a strongly-typed Java object — cleaner, supports validation, and avoids repeating `@Value("${...}")` dozens of times.

40.2 Real Example

```
# application.yml
bank:
  loan:
    max-amount: 5000000
    min-credit-score: 650
    interest-rate: 8.5
    allowed-types: [ PERSONAL, HOME, AUTO ]

@Component
@ConfigurationProperties(prefix = "bank.loan")
@Validated
@Data // Lombok – generates getters/setters, required for property binding to work
public class LoanProperties {
    @NotNull @Positive
    private BigDecimal maxAmount;

    @Min(300) @Max(900)
    private int minCreditScore;

    private double interestRate;
    private List<String> allowedTypes;
}

// Usage – inject the whole strongly-typed group wherever needed
@Service
@RequiredArgsConstructor
public class LoanService {
    private final LoanProperties loanProperties;

    public boolean isEligible(BigDecimal amount, int creditScore) {
        return amount.compareTo(loanProperties.getMaxAmount()) <= 0
            && creditScore >= loanProperties.getMinCreditScore();
    }
}
```

Real scenario: "Our Loans Service uses one `@ConfigurationProperties` class for every business rule threshold (max amount, min credit score, interest rate bands) — when the business team changes a lending policy, we update `application-prod.yml` via our Config Server (Microservices guide, Section 17) and refresh, with zero code changes or redeployment needed."

41. Logging in Spring Boot

Full deep-dive on logging fundamentals (SLF4J, Logback, Log4j2, log levels, structured logging, correlation IDs) is covered extensively in the Observability & Monitoring guide — this section focuses specifically on Spring Boot's built-in logging configuration conventions.

41.1 Spring Boot's Default Logging Setup

```
# application.yml — Spring Boot's built-in logging configuration shortcuts
logging:
  level:
    root: INFO
    com.bank.accounts: DEBUG      # fine-grained per-package level
    org.hibernate.SQL: DEBUG      # see the actual generated SQL — useful for debugging N+1 issues
  file:
    name: logs/accounts-service.log
  pattern:
    console: "%d{HH:mm:ss} %-5level [%thread] %logger{36} - %msg%n"
```

41.2 Real Scenario — Enabling SQL Logging Temporarily in Production

```
# Using Actuator's /actuator/loggers endpoint (Section 4.4, Observability guide) — no redeploy needed
POST /actuator/loggers/org.hibernate.SQL
{ "configuredLevel": "DEBUG" }

# Remember to set it back to a normal level afterward — SQL logging at scale is expensive
```

42. Spring Transactions — @Transactional Deep Dive

42.1 What a Transaction Is (Recap)

A transaction is a unit of work that must either fully succeed (commit) or fully fail (rollback) as one atomic operation — e.g., "debit account A and credit account B" must never partially apply.

42.2 How @Transactional Actually Works — AOP Proxies

Just like other AOP-based features (Section 20), `@Transactional` works via a Spring-generated proxy wrapping your bean — when a transactional method is called externally, the proxy opens a transaction, calls your real method, then commits or rolls back based on the outcome. This is exactly why the "self-invocation" gotcha (Section 20.5) applies to `@Transactional` too.

42.3 Real Code

```
@Service
@RequiredArgsConstructor
public class TransferService {
    private final AccountRepository accountRepository;

    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amount) {
        Account from = accountRepository.findById(fromId).orElseThrow();
        Account to = accountRepository.findById(toId).orElseThrow();

        from.debit(amount);
        to.credit(amount);

        accountRepository.save(from);
        accountRepository.save(to);
        // if ANY exception is thrown anywhere above, EVERYTHING here rolls back automatically
    }
}
```

42.4 Key @Transactional Attributes

Attribute	Purpose
<code>propagation</code>	How this transaction relates to an existing one, if any (Section 43)
<code>isolation</code>	How isolated this transaction is from other concurrent transactions (Section 44)
<code>readOnly</code>	Hints the underlying driver/Hibernate this is a read-only transaction, enabling performance optimizations
<code>timeout</code>	Maximum seconds the transaction may run before being forcibly rolled back

`rollbackFor` /
`noRollbackFor`

Explicitly controls which exceptions trigger (or don't trigger) a rollback (Section 45)

```
@Transactional(readOnly = true, timeout = 5)
public List<AccountDto> getAllAccounts() { ... } // read-only queries – performance hint, no write-lock overhead
```

42.5 Where to Put @Transactional — Real Practice

Best practice: Put `@Transactional` on **Service** layer methods, never on Controllers (HTTP concerns shouldn't dictate transaction boundaries) and never on Repository methods individually (a single business operation often spans multiple repository calls that must succeed/fail together, as shown in Section 42.3).

43. Transaction Propagation

43.1 What Propagation Controls

Propagation defines what happens when a `@Transactional` method is called from within another method that's already running inside a transaction — does it join the existing one, or start a completely separate one?

43.2 The Propagation Types

Propagation	Behavior	Real Use Case
REQUIRED (default)	Joins the existing transaction if one exists, otherwise creates a new one	The vast majority of service methods — normal business operations
REQUIRES_NEW	Always suspends any existing transaction and starts a brand-new, independent one	Audit logging that must be saved even if the main business transaction later rolls back
NESTED	Creates a savepoint within the existing transaction — can roll back to that savepoint without rolling back the whole outer transaction	A sub-step that might fail and needs partial rollback without aborting everything
SUPPORTS	Joins an existing transaction if present, otherwise runs non-transactionally	Read operations that don't strictly require a transaction but can use one if available
NOT_SUPPORTED	Suspends any existing transaction and runs without one	Rare — a long-running read that shouldn't hold a DB transaction/lock open
MANDATORY	Must run within an existing transaction — throws an exception if none exists	Enforcing that a method is never called standalone, only as part of a larger transactional flow
NEVER	Must NOT run within a transaction — throws an exception if one exists	Rare edge cases

43.3 Real Example — REQUIRES_NEW for Audit Logging

```
@Service
@RequiredArgsConstructor
public class LoanService {
    private final AuditService auditService;

    @Transactional
    public void approveLoan(Long loanId) {
        Loan loan = loanRepository.findById(loanId).orElseThrow();
        loan.approve();
        loanRepository.save(loan);

        auditService.logApproval(loanId); // runs in its OWN independent transaction

        if (someRiskCheckFails()) {
            throw new RuntimeException("Risk check failed after approval"); // rolls back the loan
        }
    }
}

@Service
public class AuditService {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void logApproval(Long loanId) {
        auditLogRepository.save(new AuditLog("LOAN_APPROVED", loanId));
        // ...but this audit record is ALREADY COMMITTED in its own transaction – it survives the rollback
    }
}
```

REQUIRES_NEW vs REQUIRED — Visual Difference

Outer transaction rolls back

→ **REQUIRED: inner also rolls back**

Outer transaction rolls back

→ **REQUIRES_NEW: inner already committed independently, survives**

44. Transaction Isolation Levels

44.1 What Isolation Controls

Isolation defines how much one transaction can "see" the uncommitted or concurrently-changing data of another transaction running at the same time — a direct tradeoff between data consistency and performance/concurrency.

44.2 The Isolation Levels and the Problems They Prevent

Isolation Level	Prevents	Still Allows
READ_UNCOMMITTED	Nothing	Dirty reads, non-repeatable reads, phantom reads (rarely used in practice)
READ_COMMITTED (common default, e.g., PostgreSQL)	Dirty reads	Non-repeatable reads, phantom reads
REPEATABLE_READ (MySQL's default)	Dirty reads, non-repeatable reads	Phantom reads (mostly, depends on DB implementation)
SERIALIZABLE	Dirty reads, non-repeatable reads, phantom reads (fully serial-equivalent execution)	Nothing — strongest, but slowest and most lock-heavy

44.3 The Three Core Anomalies, Explained Simply

- **Dirty Read:** Transaction A reads data that Transaction B has changed but not yet committed — if B then rolls back, A read data that never really existed.
- **Non-Repeatable Read:** Transaction A reads the same row twice, but gets different values because Transaction B updated and committed that row in between A's two reads.
- **Phantom Read:** Transaction A runs the same query twice, but gets a different **number of rows** because Transaction B inserted/deleted a matching row in between.

44.4 Real Code — Setting Isolation Explicitly

```
@Transactional(isolation = Isolation.SERIALIZABLE)
public void transferWithStrictConsistency(Long fromId, Long toId, BigDecimal amount) {
    // used sparingly – e.g., for a critical financial reconciliation step where correctness
    // matters more than throughput, since SERIALIZABLE can cause more lock contention/retries
}
```

Real scenario: "Our standard account transfer method runs at the database's default READ_COMMITTED isolation for good throughput. Our monthly interest calculation batch job — which reads and aggregates across the entire accounts table and absolutely cannot tolerate phantom reads — explicitly uses SERIALIZABLE isolation, accepting the performance cost for correctness during that one critical process."

45. Rollback Handling

45.1 The Default Rollback Rule (A Very Common Interview Trap)

Critical default behavior: By default, Spring only rolls back a transaction automatically for **unchecked exceptions** (RuntimeException and its subclasses, plus Error) — **checked exceptions do NOT trigger a rollback by default**. This surprises many developers coming from a "any exception should roll back" assumption.

45.2 Real Example of the Trap

```
@Transactional
public void processPayment(PaymentRequest request) throws PaymentGatewayException {    // CHECKED exception
    paymentRepository.save(new Payment(request));
    if (gatewayCallFails()) {
        throw new PaymentGatewayException("Gateway timeout");    // checked – does NOT roll back by default!
    }
    // the Payment record above STAYS COMMITTED even though the gateway call failed – a real bug waiting to
    happen
}
```

45.3 The Fix — rollbackFor

```
@Transactional(rollbackFor = PaymentGatewayException.class)    // now explicitly rolls back for this checked
exception too
public void processPayment(PaymentRequest request) throws PaymentGatewayException { ... }

// Or the inverse – prevent rollback for a specific exception that's expected/non-fatal
@Transactional(noRollbackFor = MinorValidationWarning.class)
public void processWithWarnings() { ... }
```

45.4 Programmatic Rollback (Manual Control)

```
@Transactional
public void conditionalProcess() {
    if (someBusinessConditionFails()) {
        TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();    // force a rollback without
        throwing an exception
        return;
    }
    // normal processing continues
}
```

Interview tip: Always explicitly review whether your custom exceptions are checked or unchecked, and use **rollbackFor** deliberately rather than assuming Spring's default rollback behavior matches your business expectations — this exact gotcha has caused real production data-consistency bugs at many organizations.

46. Distributed Transactions

46.1 Why Local @Transactional Isn't Enough Across Microservices

Everything in Sections 42-45 works for a single database within a single service. Once a business operation spans multiple microservices with separate databases (e.g., Loans Service + Accounts Service), there's no single local transaction that can span both — this is exactly the problem the Saga Pattern (Microservices guide, Section 22) solves.

46.2 Two-Phase Commit (2PC) — The Older Approach

2PC uses a transaction coordinator to ask all participants to "prepare" (lock resources, ready to commit), then tells everyone to actually "commit" only if all participants confirmed readiness — technically possible via JTA (Java Transaction API) but rarely used in modern microservices due to blocking behavior, tight coupling, and poor availability characteristics (if the coordinator or any participant is down, everything blocks).

46.3 The Modern Approach — Saga Pattern (Recap)

Full detail already in the Microservices guide (Section 22) — break the distributed operation into a sequence of local `@Transactional` steps, each in its own service, with compensating actions to undo earlier steps if a later one fails, trading strict atomicity for eventual consistency with guaranteed rollback logic.

Local @Transactional vs Distributed Saga

Single Service: @Transactional handles atomicity automatically within one DB

— VS —

Loans Service: @Transactional (local step 1) →

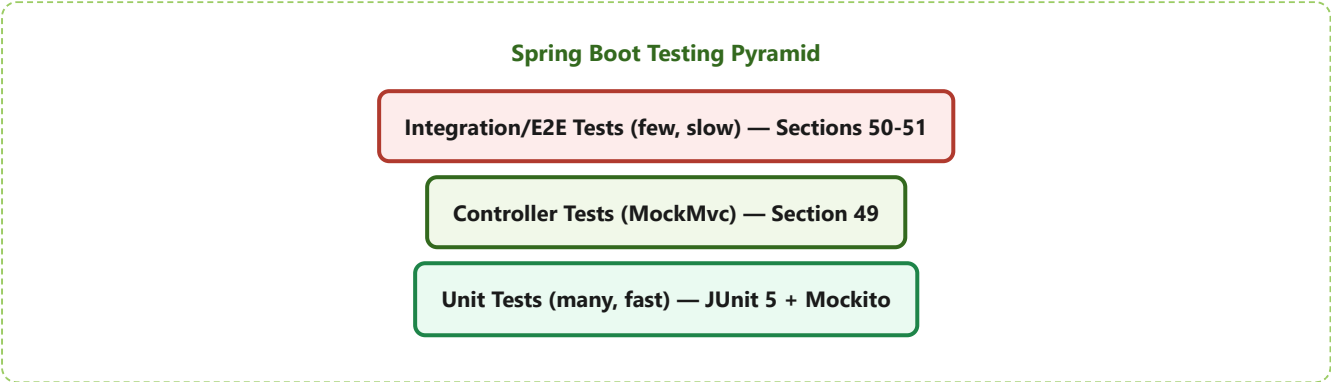
Accounts Service: @Transactional (local step 2) →

Compensating @Transactional method if step 2 fails

Interview tip: "@Transactional guarantees atomicity within a single database/service. Across microservices, we don't try to force a distributed transaction — we use the Saga pattern, where each service's own local @Transactional method handles its own piece, and a compensating transaction undoes it if a later step in the saga fails."

47. Spring Testing — JUnit 5

47.1 The Testing Pyramid Recap



47.2 Real JUnit 5 Unit Test

```
class AccountServiceTest {

    @Test
    void shouldReturnAccountWhenExists() {
        // arrange, act, assert – the standard test structure
        AccountRepository mockRepo = mock(AccountRepository.class);
        when(mockRepo.findById(1001L)).thenReturn(Optional.of(new Account(1001L, BigDecimal.TEN)));

        AccountService service = new AccountService(mockRepo);
        AccountDto result = service.findById(1001L);

        assertThat(result.balance()).isEqualByComparingTo(BigDecimal.TEN);
    }

    @Test
    void shouldThrowWhenAccountNotFound() {
        AccountRepository mockRepo = mock(AccountRepository.class);
        when(mockRepo.findById(anyLong())).thenReturn(Optional.empty());
        AccountService service = new AccountService(mockRepo);

        assertThrows(AccountNotFoundException.class, () -> service.findById(9999L));
    }

    @ParameterizedTest
    @ValueSource(strings = {"", " ", "  "})
    void shouldRejectBlankCustomerNames(String blankName) {
        assertThrows(ValidationException.class, () -> service.createAccount(blankName, BigDecimal.TEN));
    }
}
```

47.3 Key JUnit 5 Annotations

Annotation	Purpose
@Test	Marks a method as a test case

<code>@BeforeEach</code> / <code>@AfterEach</code>	Runs before/after every single test method
<code>@BeforeAll</code> / <code>@AfterAll</code>	Runs once before/after all tests in the class (must be static)
<code>@ParameterizedTest</code>	Runs the same test logic with multiple different input values
<code>@DisplayName</code>	Human-readable test name shown in reports
<code>@Disabled</code>	Temporarily skips a test (use sparingly, with a comment explaining why)

48. Mockito

48.1 Why Mocking Matters

A true unit test should test ONE class in isolation — Mockito lets you create fake ("mock") versions of a class's dependencies, so you can test business logic without needing a real database, a real network call, or a real third-party API.

48.2 Real Mockito Test — Cleaner With Annotations

```
@ExtendWith(MockitoExtension.class)
class LoanServiceTest {

    @Mock
    private LoanRepository loanRepository;

    @Mock
    private CreditBureauClient creditBureauClient;

    @InjectMocks // Mockito automatically injects the @Mock fields above into this real object
    private LoanService loanService;

    @Test
    void shouldApproveLoanWhenCreditScoreIsGood() {
        when(creditBureauClient.getScore("CUST-1")).thenReturn(new CreditScore(750));
        when(loanRepository.save(any(Loan.class))).thenAnswer(inv -> inv.getArgument(0));

        LoanResponse response = loanService.applyForLoan("CUST-1", BigDecimal.valueOf(500000));

        assertThat(response.getStatus()).isEqualTo("APPROVED");
        verify(loanRepository, times(1)).save(any(Loan.class)); // verify a specific interaction happened
    }

    @Test
    void shouldRejectLoanWhenCreditBureauThrowsException() {
        when(creditBureauClient.getScore(anyString())).thenThrow(new CreditBureauException("timeout"));

        LoanResponse response = loanService.applyForLoan("CUST-2", BigDecimal.valueOf(500000));

        assertThat(response.getStatus()).isEqualTo("REJECTED_MANUAL_REVIEW"); // verify graceful fallback
        behavior
    }
}
```

48.3 Key Mockito Concepts

Concept	Purpose
<code>@Mock</code>	Creates a fake implementation of a dependency
<code>@InjectMocks</code>	Creates the real object under test, auto-wiring the <code>@Mock</code> fields into it
<code>when(...).thenReturn(...)</code>	Stubs a mock's behavior for a given input

<code>verify(...)</code>	Asserts that a specific method WAS (or wasn't) called, and how many times
<code>ArgumentCaptor</code>	Captures the actual argument passed to a mock, for deeper assertions
<code>@Spy</code>	Wraps a REAL object but lets you selectively override specific method behaviors

49. MockMvc

49.1 What MockMvc Tests

MockMvc lets you test the full Spring MVC request-handling pipeline (routing, controller method invocation, validation, JSON serialization, exception handling) WITHOUT starting a real HTTP server — much faster than a full end-to-end test, while still testing more than a pure unit test would.

49.2 Real @WebMvcTest — Slice Testing the Controller Layer

```
@WebMvcTest(AccountController.class) // only loads the web layer, NOT the full application context – fast
class AccountControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean // replaces the real AccountService bean with a Mockito mock inside the Spring context
    private AccountService accountService;

    @Test
    void shouldReturnAccountJson() throws Exception {
        when(accountService.findById(1001L)).thenReturn(new AccountDto(1001L, BigDecimal.valueOf(4500),
"ACTIVE"));

        mockMvc.perform(get("/api/v1/accounts/1001"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.balance").value(4500))
            .andExpect(jsonPath("$.status").value("ACTIVE"));
    }

    @Test
    void shouldReturn404WhenAccountNotFound() throws Exception {
        when(accountService.findById(9999L)).thenReturn(new AccountNotFoundException(9999L));

        mockMvc.perform(get("/api/v1/accounts/9999"))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.message").value(containsString("9999")));
    }

    @Test
    void shouldReturn400WhenRequestBodyInvalid() throws Exception {
        mockMvc.perform(post("/api/v1/accounts")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"initialDeposit\": -100 }")) // invalid – negative deposit, missing customerName
            .andExpect(status().isBadRequest());
    }
}
```

Interview tip: `@WebMvcTest` only loads Controllers, filters, and MVC-related infrastructure (a "slice" of the full context) — it deliberately does NOT load `@Service/@Repository` beans, which is exactly why you need `@MockBean` to fake the service layer, making these tests much faster than a full `@SpringBootTest`.

50. Integration Testing in Spring Boot

50.1 What Integration Tests Cover That Unit Tests Don't

Integration tests load the REAL, full Spring application context (or a large slice of it) and test how components actually work together — including real database interaction, real Spring Data JPA query generation, and real bean wiring — catching issues that mocked unit tests can miss entirely.

50.2 Real @SpringBootTest

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureMockMvc
class AccountIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private AccountRepository accountRepository; // the REAL repository, backed by a real (test) database

    @Test
    void shouldCreateAndRetrieveAccount() throws Exception {
        mockMvc.perform(post("/api/v1/accounts")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"customerName\":\"John\",\"initialDeposit\":1000}"))
            .andExpect(status().isCreated());

        assertThat(accountRepository.findAll()).hasSize(1); // verifying REAL data actually persisted to the
        database
    }
}
```

Real problem this avoids: A pure unit test with a mocked repository can pass even if your actual JPQL query has a typo or your entity mapping is wrong — because the mock never really executes SQL. Integration tests catch these real wiring/query bugs that unit tests structurally cannot.

51. TestContainers

51.1 The Problem Testcontainers Solves

Integration tests need a real database — but using an in-memory database (H2) instead of your real production database (PostgreSQL/MySQL) risks subtle SQL dialect differences that hide real bugs until production. Testcontainers spins up a real, throwaway Docker container of your actual database engine, just for the test run.

51.2 Real Testcontainers Code (Recap From Microservices Guide, Section 19.3)

```
@SpringBootTest
@Testcontainers
class OrderRepositoryIntegrationTest {

    @Container
    static PostgreSQLContainer<> postgres = new PostgreSQLContainer<>("postgres:15")
        .withDatabaseName("accounts_test");

    @DynamicPropertySource
    static void configureProps(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired
    private AccountRepository accountRepository;

    @Test
    void savesAndFetchesAccount() {
        Account saved = accountRepository.save(new Account("Jane Doe", BigDecimal.valueOf(2500)));
        assertThat(accountRepository.findById(saved.getId()).isPresent());
    }
}
```

51.3 Real Testcontainers for Redis Too

```
@Container
static GenericContainer<> redis = new GenericContainer<>("redis:7-alpine").withExposedPorts(6379);

@DynamicPropertySource
static void redisProps(DynamicPropertyRegistry registry) {
    registry.add("spring.data.redis.host", redis::getHost);
    registry.add("spring.data.redis.port", () -> redis.getMappedPort(6379));
}
```

Interview tip: Testcontainers gives you the confidence of testing against the exact same database/cache engine used in production, without needing a shared, always-on test database — every test run gets a fresh, isolated, disposable container, eliminating "tests pass locally but fail in CI due to leftover data" issues entirely.

52. Caching in Spring Boot — Complete Deep Dive

52.1 Why Caching Matters (Recap)

Repeated calls for the same rarely-changing data (product catalogs, interest rate tables, customer profile lookups) hitting the database every single time wastes resources and adds latency — caching stores the result of an expensive operation so subsequent identical calls return instantly from memory.

52.2 Enabling Caching in Spring Boot

```
@SpringBootApplication
@EnableCaching // turns on Spring's caching abstraction
public class BankApplication { ... }
```

52.3 The Core Caching Annotations

Annotation	Purpose
<code>@Cacheable</code>	Caches the method's return value; on a cache hit, the method body is skipped entirely
<code>@CachePut</code>	Always executes the method AND updates the cache with the new result (used for updates)
<code>@CacheEvict</code>	Removes an entry (or all entries) from the cache — used after deletes/updates that invalidate cached data
<code>@Caching</code>	Combines multiple caching annotations on one method
<code>@CacheConfig</code>	Class-level shared cache configuration (e.g., default cache name) for all methods in that class

52.4 Real Code — Full Cache Lifecycle

```

@Service
@CacheConfig(cacheNames = "accounts")
@RequiredArgsConstructor
public class AccountService {
    private final AccountRepository accountRepository;

    @Cacheable(key = "#id")
    public AccountDto findById(Long id) {
        log.info("Cache MISS – hitting the database for account {}", id); // only logs on an actual cache
miss
        return mapper.toDto(accountRepository.findById(id).orElseThrow());
    }

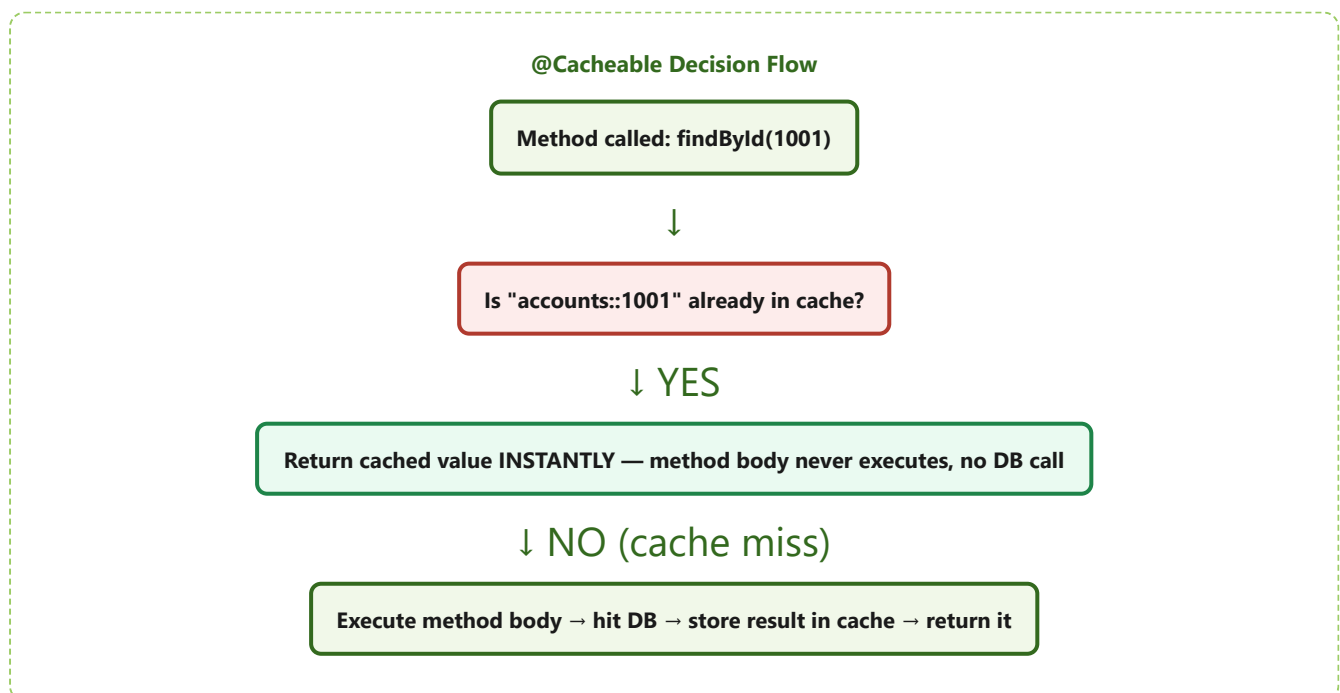
    @CachePut(key = "#result.id") // always runs, then refreshes the cache with the updated value
    public AccountDto updateAccount(Long id, UpdateAccountRequest request) {
        Account account = accountRepository.findById(id).orElseThrow();
        account.updateFrom(request);
        return mapper.toDto(accountRepository.save(account));
    }

    @CacheEvict(key = "#id") // removes the stale entry so the next read is a fresh cache miss
    public void closeAccount(Long id) {
        accountRepository.deleteById(id);
    }

    @CacheEvict(allEntries = true) // nukes the ENTIRE "accounts" cache – used sparingly, e.g., after a bulk
import job
    public void evictAllAccountsCache() { }
}

```

52.5 Diagram — @Cacheable Flow



52.6 Cache Providers Spring Boot Supports

Provider	Notes
Simple (ConcurrentHashMap, default)	In-memory, per-instance only — fine for local dev, NOT suitable for multi-instance production (each pod has its own separate cache, causing inconsistency)
Caffeine	High-performance in-memory Java cache with size/TTL eviction — still per-instance, good for single-instance or read-mostly local caching
Redis	External, shared, distributed cache — the standard real-world choice for multi-instance microservices (Section 53)
EhCache	Older, mature caching library, still seen in legacy applications

Critical real-world gotcha: The default Simple/ConcurrentHashMap cache is **per-JVM-instance** — if you run 4 replicas of Accounts Service behind a load balancer, each pod has its OWN separate cache, meaning a cache update on pod 1 is invisible to pods 2-4. Any real multi-instance microservice **MUST** use an external shared cache like Redis, or risk serving stale/inconsistent cached data depending on which pod handles the request.

52.7 Cache Key Design (Using SpEL — Recap From Section 19)

```
@Cacheable(key = "#customerId + '-' + #accountType") // composite key from multiple parameters
public List<AccountDto> findAccounts(Long customerId, String accountType) { ... }

@Cacheable(condition = "#customerId != null", unless = "#result == null") // conditional caching
public AccountDto findById(Long customerId) { ... }
```

53. Redis with Spring Boot — Complete Deep Dive

53.1 What Redis Is

Redis is an in-memory key-value data store — extremely fast (sub-millisecond) reads/writes, supporting rich data structures (strings, hashes, lists, sets, sorted sets) — the most common real-world choice for distributed caching, session storage, rate limiting, and even simple message queuing (Pub/Sub).

53.2 Real Dependency and Configuration

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>

# application.yml
spring:
  data:
    redis:
      host: redis-cluster.prod.internal
      port: 6379
      password: ${REDIS_PASSWORD}      # from Azure Key Vault / Variable Group, never hardcoded# default TTL:
1 hour, in milliseconds
```

53.3 Redis as Spring's Cache Provider (Connects Directly to Section 52)

```
@Configuration
public class RedisConfig {

    @Bean
    public RedisCacheConfiguration cacheConfiguration() {
        return RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(30))
            .serializeValuesWith(RedisSerializationContext.SerializationPair
                .fromSerializer(new GenericJackson2JsonRedisSerializer())); // store cached objects as JSON,
not raw Java serialization
    }

    @Bean
    public CacheManager cacheManager(RedisConnectionFactory factory, RedisCacheConfiguration config) {
        return RedisCacheManager.builder(factory)
            .cacheDefaults(config)
            .withCacheConfiguration("accounts", config.entryTtl(Duration.ofHours(1))) // per-cache-name
overrides
            .withCacheConfiguration("productCatalog", config.entryTtl(Duration.ofDays(1)))
            .build();
    }
}

// The SAME @Cacheable annotations from Section 52 now transparently use Redis instead of local memory
@Cacheable(cacheNames = "accounts", key = "#id")
public AccountDto findById(Long id) { ... }
```

53.4 Using Redis Directly — RedisTemplate (Beyond Simple Caching)

```
@Service
@RequiredArgsConstructor
public class SessionTokenService {
    private final RedisTemplate<String, String> redisTemplate;

    public void storeToken(String sessionId, String token) {
        redisTemplate.opsForValue().set("session:" + sessionId, token, Duration.ofMinutes(15)); // with TTL
    }

    public String getToken(String sessionId) {
        return redisTemplate.opsForValue().get("session:" + sessionId);
    }

    public void incrementLoginAttempts(String username) {
        redisTemplate.opsForValue().increment("login-attempts:" + username); // atomic counter – great for
rate limiting
    }
}
```

53.5 Real Redis Use Cases in Our Organization

Use Case	Redis Feature Used
API response caching	Spring Cache abstraction backed by Redis (Section 52.6/53.3)
Rate limiting (API Gateway)	Atomic counters + TTL (Microservices guide, Section 20)
Session/token storage	Key-value with expiry, shared across all service instances
Distributed locks	Redisson/Redis SETNX pattern — preventing duplicate concurrent processing across multiple pods
Leaderboards/rankings	Redis Sorted Sets (ZSET) for fast ranked queries
Pub/Sub for lightweight real-time notifications	Redis Pub/Sub channels (not a replacement for Kafka's durability, but fine for ephemeral fan-out)

53.6 Redis Eviction Policies (What Happens When Redis Runs Out of Memory)

Policy	Behavior
noeviction	Rejects new writes once memory is full — errors instead of losing data (risky for a pure cache use case)
allkeys-lru	Evicts the Least Recently Used key across all keys — the most common choice for a pure cache
volatile-lru	Evicts LRU only among keys that have a TTL set, leaving permanent keys untouched
allkeys-random	Evicts a random key — rarely the best choice, but very cheap to compute

53.7 Redis Cache Invalidation — The Hardest Problem in Caching

"There are only two hard things in computer science: cache invalidation and naming things." Real bugs happen when an update path forgets to evict/refresh the cache — e.g., updating a card limit directly via a batch SQL script bypasses `@CachePut` / `@CacheEvict` entirely, leaving Redis serving a stale limit until the TTL expires.

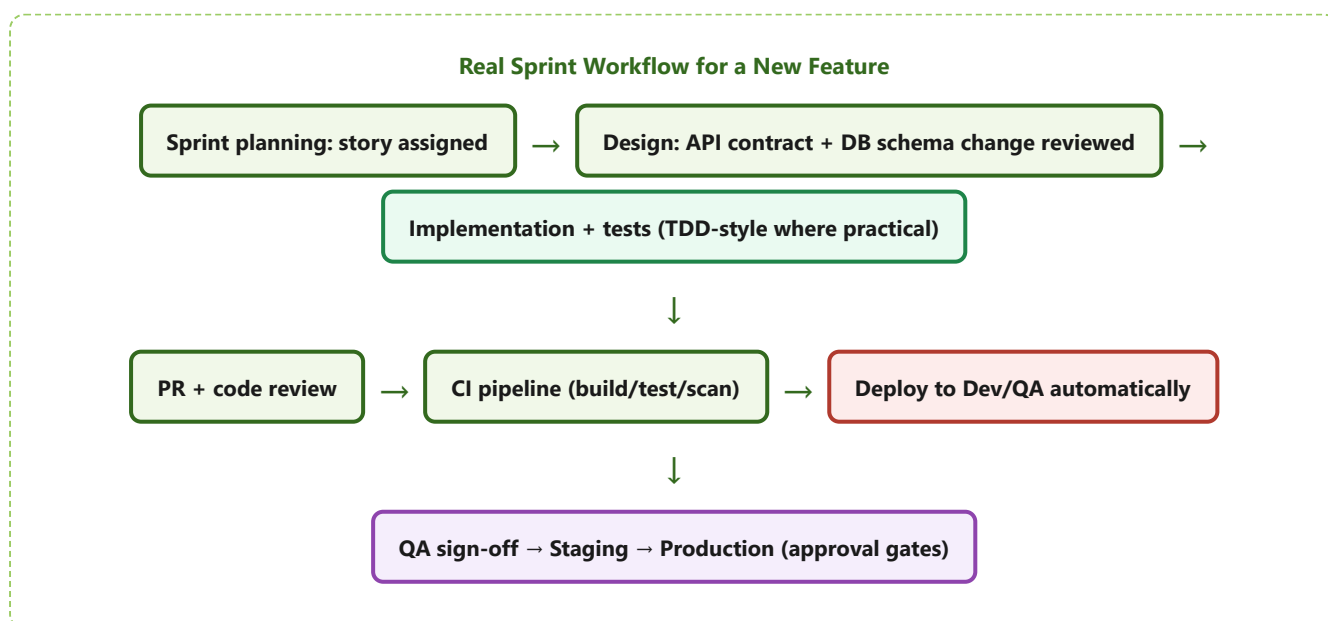
Real scenario: "A production incident once involved a customer seeing an outdated card limit after our support team manually corrected a data issue directly in the database — bypassing the application layer entirely meant Redis never got evicted. We fixed the process by requiring all manual data corrections to go through an authenticated internal API endpoint that properly triggers cache eviction, never direct SQL against production."

54. How We Develop Spring Boot Applications in Real-Time (Organization Level)

54.1 The Full Development Lifecycle

1. **Requirement & API contract design** — agree on the OpenAPI spec with frontend/consuming teams before coding (API guide, Section 39).
2. **Scaffold the project** using Spring Initializr with the org's standard starter dependencies and package structure (Section 11).
3. **Set up the database** — create migration scripts (Flyway/Liquibase), never rely on `ddl-auto: update` in production.
4. **Implement layer by layer** — Entity → Repository → Service (with `@Transactional` boundaries) → DTO/Mapper → Controller (with validation) → GlobalExceptionHandler.
5. **Add caching** where read-heavy, rarely-changing data justifies it (Sections 52-53).
6. **Add resilience** for any outbound calls (Resilience4j — Microservices guide Section 18).
7. **Write tests as you go** — unit tests (Mockito) alongside the service logic, MockMvc tests for the controller layer, integration tests for critical flows.
8. **Document via Swagger/OpenAPI annotations** directly in the controller code (API guide, Section 16).
9. **Local verification** — run the full stack via `docker compose up` (Docker & Kubernetes guide, Section 10) including Postgres/Redis/Kafka.
10. **Push to a feature branch, open a PR** — CI pipeline runs build, unit tests, integration tests (Testcontainers), SonarQube quality gate, security scan.
11. **Code review + merge** — at least 2 reviewers, linked work item (Git/CI-CD guide conventions).
12. **Automated deploy** to Dev → QA → Staging (with approval) → Production (with approval), per the CI/CD guide's Azure DevOps pipeline pattern.
13. **Monitor in production** via Actuator/Micrometer/Grafana dashboards specific to this service's Golden Signals (Observability guide).

54.2 Real Sprint-Level Workflow (Agile Team Practice)



54.3 Code Review Standards Real Teams Enforce

- Constructor injection only, no field injection (Section 13).
- No entities returned directly from controllers — DTOs mandatory (API guide, Section 9).
- Every new outbound call must have a timeout and, where appropriate, a circuit breaker.
- Every new endpoint must have Swagger annotations and at least one MockMvc test.
- No hardcoded secrets/config — must come from application.yml/Config Server/Key Vault.
- SonarQube quality gate must pass (coverage threshold, no new critical code smells).

55. Common Spring/Spring Boot Issues, Problems & Solutions

Issue	Cause	Real Solution
"No qualifying bean of type X" at startup	Missing @Component/@Service annotation, or bean not in a scanned package	Verify annotation exists, check @ComponentScan base package covers the class's package
Circular dependency exception	Bean A depends on Bean B which depends back on Bean A	Refactor to break the cycle (extract shared logic into a third bean), or use @Lazy as a last resort
@Transactional method doesn't roll back on exception	Checked exception thrown without rollbackFor, or self-invocation bypassing the proxy	Use rollbackFor explicitly (Section 45); move self-invoked method to a separate bean (Section 20.5)
LazyInitializationException	Accessing a lazy-loaded JPA relationship outside an active transaction/session (e.g., in the controller after the service method returned)	Map to DTOs within the transactional service method, or use JOIN FETCH/entity graphs (API guide, Section 45)
Application works locally but fails with a different cache result across pods	Using the default per-JVM cache (ConcurrentHashMap) in a multi-instance deployment	Switch to Redis as the shared cache backend (Section 52.6)
Bean initialized before its dependency is ready ("chicken and egg")	Complex @PostConstruct ordering issues	Use ApplicationListener<ContextRefreshedEvent> or restructure dependencies instead of relying on init order
Slow application startup	Too much component scanning, heavy @PostConstruct logic, large auto-configuration surface	Narrow @ComponentScan base packages, move heavy startup work to lazy/async initialization
@Value injection returns null	Property genuinely missing from active profile, or class not a Spring-managed bean at all	Verify property exists in the active profile's yml; ensure the class is annotated as a bean (not manually "newed")
Tests pass locally but fail in CI	Reliance on local machine state (H2 instead of real DB, hardcoded ports, timezone assumptions)	Use Testcontainers (Section 51) for real-database parity, avoid environment-dependent assumptions
MockMvc test fails with 403/401 unexpectedly	Spring Security auto-configuration active in the test slice, blocking the request	Add @WithMockUser for the test, or explicitly configure/mock the security context for @WebMvcTest

56. Most Important Interview Topics — Deep Dive Recap

These are the topics interviewers ask about most frequently for Spring/Spring Boot roles — cross-referenced back to their full explanation sections. Make sure you can explain each one confidently, ideally while drawing the relevant diagram.

Topic	Section	One-Line Recap
Dependency Injection	13	Container creates/injects dependencies instead of your code manually instantiating them
IoC Container	12	ApplicationContext creates, wires, and manages the full lifecycle of all beans
Bean Lifecycle	14	Instantiate → Populate Properties → Aware Interfaces → @PostConstruct → In Use → @PreDestroy
AOP	20	Cross-cutting concerns applied via proxies without cluttering business logic
Spring MVC Request Flow	21	DispatcherServlet → HandlerMapping → Controller → ViewResolver → View
Auto Configuration	33	Classpath-driven, conditional bean registration — always overridable
Profiles	36	Same JAR, different config per environment via active profile
Exception Handling	14 (API guide), 30	@RestControllerAdvice + @ExceptionHandler for a consistent global error response

57. Interview Questions & Answers (Basic → Advanced → Scenario Based)

Q1. What problem does Spring solve, and why was it created?

Spring was created as a lightweight alternative to heavyweight EJB, using Dependency Injection and plain POJOs to provide enterprise features (transactions, AOP) without requiring an EJB container or framework-specific base classes — making applications simpler to build and unit test.

Q2. What is the difference between Spring, Spring MVC, and Spring Boot?

Spring (Core) provides the IoC container and dependency injection; Spring MVC is a web framework built on top of Spring Core; Spring Boot is a convention-over-configuration layer on top of both, adding auto-configuration, starter dependencies, and embedded servers — they're layered, not competing.

Q3. Explain the three types of Dependency Injection and which is recommended.

Constructor, setter, and field injection. Constructor injection is recommended — it makes dependencies explicit and immutable (final fields), fails fast at startup if something's missing, and makes unit testing trivial without needing Spring or reflection.

Q4. Walk through the Spring Bean lifecycle.

Instantiate (constructor called) → Populate Properties (DI happens) → Aware interfaces invoked → @PostConstruct/InitializingBean (custom init logic) → bean is ready and in use → @PreDestroy/DisposableBean during shutdown for cleanup.

Q5. What's the difference between singleton and prototype bean scope, and what's a common gotcha?

Singleton creates exactly one shared instance per container (the default); prototype creates a new instance every time it's requested. The gotcha: injecting a prototype bean into a singleton via normal injection only creates it once at singleton-creation time — you need ObjectProvider or a scoped proxy to actually get fresh instances per use.

Q6. Explain AOP and how Spring implements it.

AOP lets you define cross-cutting concerns (logging, auditing, transactions) once as an "Aspect" and apply them declaratively via Pointcuts, instead of repeating the logic in every method. Spring implements this via runtime proxies wrapping your beans — calling a method actually calls the proxy first, which runs the aspect logic, then delegates to your real method.

Q7. Walk through the complete Spring MVC request flow.

A request hits DispatcherServlet (Front Controller) → HandlerMapping determines which controller method handles it → the Controller executes (calling the service layer, populating the Model) → it returns a logical view name → ViewResolver resolves it to an actual View → the View renders using Model data → DispatcherServlet sends the final response.

Q8. How does Spring Boot's auto-configuration actually work?

At startup, Spring Boot inspects the classpath and conditionally registers beans (via @ConditionalOnClass, @ConditionalOnMissingBean, etc.) that a typical use case would need — e.g., seeing a JPA starter and a DB driver auto-configures a DataSource and EntityManagerFactory. Defining your own bean of the same type automatically disables the auto-configured one.

Q9. What are Spring Profiles and why are they important?

Profiles let the exact same built artifact run with different configuration per environment (application-dev.yml, application-prod.yml) activated via --spring.profiles.active — critical for the "build once, promote everywhere" CI/CD principle, avoiding rebuilding the artifact per environment.

Q10. Explain @Transactional propagation, specifically REQUIRED vs REQUIRES_NEW.

REQUIRED (the default) joins an existing transaction if one is active, or creates a new one if not — if the outer transaction rolls back, everything inside it rolls back too. REQUIRES_NEW always suspends any existing transaction and starts an independent one that commits/rolls back on its own — useful for audit logs that must persist even if the main business transaction later fails.

Real scenario: Logging a loan approval audit record with REQUIRES_NEW ensures the audit trail survives even if a later risk check causes the main approval transaction to roll back.

Q11. Why doesn't Spring roll back a transaction for a checked exception by default, and how do you fix it?

Spring's default rollback rule only covers unchecked exceptions (RuntimeException/Error) — checked exceptions require explicitly specifying `rollbackFor = YourCheckedException.class` on `@Transactional`, otherwise the transaction commits even though an exception was thrown, a common real production bug.

Q12. What are the transaction isolation levels, and what problem does each prevent?

READ_UNCOMMITTED prevents nothing; READ_COMMITTED prevents dirty reads; REPEATABLE_READ additionally prevents non-repeatable reads; SERIALIZABLE prevents all three anomalies (dirty reads, non-repeatable reads, phantom reads) at the cost of the most locking/performance overhead.

Q13. How do you unit test a Spring service without starting the full application context?

Use plain JUnit 5 + Mockito — create `@Mock` objects for dependencies, `@InjectMocks` the real service class, and stub behavior with `when()/thenReturn()`. No Spring context is loaded at all, making these tests extremely fast.

Q14. What's the difference between @WebMvcTest and @SpringBootTest?

`@WebMvcTest` loads only the web/MVC layer (a "slice") for fast controller-focused testing with `MockMvc`, requiring `@MockBean` for service dependencies. `@SpringBootTest` loads the full real application context, suitable for true integration tests that need real bean wiring and database interaction (often paired with `Testcontainers`).

Q15. Why use Testcontainers instead of an in-memory database like H2 for tests?

H2 can behave subtly differently from your real production database (SQL dialect quirks, feature support gaps), hiding real bugs until production. `Testcontainers` spins up the actual database engine (e.g., real PostgreSQL) in a disposable Docker container per test run, giving much higher confidence the tests reflect real behavior.

Q16. How does Spring's caching abstraction work, and what's the danger of the default cache provider?

`@Cacheable/@CachePut/@CacheEvict` declaratively cache method results by key. The default Simple (ConcurrentHashMap) provider is per-JVM-instance — in a multi-replica deployment, each pod has its own separate cache, causing inconsistent results depending on which pod serves a request. Production multi-instance services must use a shared external cache like Redis.

Q17. What are real use cases for Redis beyond simple caching?

Rate limiting via atomic counters with TTL, session/token storage shared across instances, distributed locks (preventing duplicate concurrent processing across pods), sorted-set-based leaderboards, and lightweight Pub/Sub for ephemeral real-time notifications.

Q18. What is the difference between JPA and Hibernate?

JPA is a specification (interfaces/annotations like `@Entity`, `EntityManager`); Hibernate is a concrete implementation of that specification. Spring Boot's default underlying JPA implementation is Hibernate, but you're coding against the JPA API layer.

Q19. Why did the Java web stack evolve from Servlets → Struts → Spring → Spring Boot?

Each layer solved the previous layer's pain: Servlets enabled dynamic HTTP handling but mixed HTML and Java code; Struts added MVC structure but required heavy XML; Spring introduced lightweight DI to replace heavyweight EJB; Spring MVC added a cleaner web framework on Spring Core; Spring Boot eliminated remaining manual configuration via auto-configuration and embedded servers.

Q20. Describe a real production issue you've debugged involving Spring/Spring Boot and how you resolved it.

Example: a `LazyInitializationException` appeared intermittently in production when a controller tried to access a lazily-loaded JPA relationship after the transactional service method had already returned and closed the session. Fixed by mapping to DTOs entirely within the transactional service method boundary, ensuring all needed data is fetched while the session is still open.

58. Final Summary & Cheat Sheet

The journey: Core Java → Servlets/JSP (dynamic but messy) → JDBC (functional but boilerplate-heavy) → Hibernate/JPA (ORM solves mapping pain) → Spring Framework (lightweight DI replaces heavyweight EJB) → Spring MVC (clean web framework) → Spring Boot (auto-configuration, embedded servers, "just run it").

Spring Core: IoC container manages bean creation/wiring/lifecycle via Dependency Injection (prefer constructor injection), with AOP handling cross-cutting concerns declaratively.

Spring MVC: DispatcherServlet as Front Controller, routing through HandlerMapping to Controllers, rendering via ViewResolver (JSP/Thymeleaf) or returning JSON directly (@RestController).

Spring Boot: Auto-configuration + starters + embedded Tomcat eliminate nearly all manual setup; Profiles/YAML externalize environment-specific config; Actuator provides production-ready operational endpoints.

Transactions: @Transactional (via AOP proxies) manages atomicity within one database; Propagation controls nested transaction behavior; Isolation controls concurrent-transaction visibility; distributed operations across microservices use the Saga pattern instead of a single distributed transaction.

Testing: JUnit 5 + Mockito for fast isolated unit tests, MockMvc for controller-layer slice tests, @SpringBootTest + Testcontainers for true integration tests against real database/cache engines.

Caching/Redis: @Cacheable family for declarative caching; Redis as the mandatory shared backend for any multi-instance production deployment, also powering rate limiting, distributed locks, and session storage.

58.1 Quick Cheat Sheet

Term	One-Line Meaning
IoC Container	Manages bean creation, wiring, and lifecycle (ApplicationContext)
Dependency Injection	Container provides dependencies instead of code manually creating them
Bean	An object managed by the Spring container
AOP	Cross-cutting logic (logging, transactions) applied via proxies
DispatcherServlet	Spring MVC's single Front Controller handling every request
Auto Configuration	Classpath-driven, overridable, conditional bean setup in Spring Boot
Starter	Curated dependency bundle for a common use case
Profile	Environment-specific configuration set, same artifact everywhere
Actuator	Built-in production-ready health/metrics/ops endpoints
@Transactional	Declarative transaction boundary via AOP proxy
Propagation	How a transaction relates to an already-active one
Isolation	How much concurrent transactions can see of each other's changes
MockMvc	Fast, in-process Spring MVC layer testing without a real server
Testcontainers	Real, disposable Docker-based dependencies for integration tests

@Cacheable	Declaratively caches a method's return value
Redis	Shared, distributed in-memory store — mandatory cache backend for multi-instance apps

Use this guide alongside the Git/GitHub/GitLab/Bitbucket guide, the CI/CD guide, the Docker & Kubernetes guide, the Microservices guide, the Observability & Monitoring guide, and the API Development guide — together they cover the complete real-world Java full-stack developer journey from foundational Java web history, through building production-grade Spring Boot services, to deploying, securing, and observing them reliably at scale.